



APLICACIÓN DE REDES NEURONALES A LA RESOLUCIÓN NUMÉRICA DE ECUACIONES EN DERIVADAS PARCIALES



**Trabajo de Fin de Grado
Matemáticas**

Alberto Nieto Cardoso

Dirigido por Norberto Octavio Corral Blanco

Codirigido por Carlos Enrique Carleos Arttime



Universidad de Oviedo

Índice

1	Introducción	1
2	Teoría Fundamental de Ecuaciones en Derivadas Parciales	3
2.1	Nociones básicas	3
2.2	La ecuación del calor	9
2.3	La ecuación de Poisson	14
2.4	Las ecuaciones de Navier-Stokes	17
3	Formalización del Método de Diferencias Finitas	25
3.1	Definiciones y resultados clave	25
3.2	Particularización y tratamiento de errores	27
4	Conceptos Básicos de Machine Learning	43
4.1	Fundamentos	43
4.2	El perceptrón multicapa	45
4.3	Implementación en el campo de las EDPs	48
4.4	Comentarios y variantes del modelo	49
5	Resultados	55
5.1	Comparativa entre redes neuronales	55
5.2	Diferencias entre métodos clásicos y modernos	60
5.3	Discusión	71
6	Conclusiones	75
7	Bibliografía	77
8	Glosario	79

Capítulo 1

INTRODUCCIÓN

Las ecuaciones en derivadas parciales son un tipo particular de ecuaciones que juegan un papel fundamental en la modelización de diversos fenómenos, siendo especialmente importantes en física, ingeniería y meteorología. De entre los muchos problemas famosos que existen, la modelización de un fluido mediante el uso de las ecuaciones de Navier-Stokes destaca tanto por su complejidad como por su utilidad.

Actualmente, estos problemas se resuelven en la mayoría de casos mediante métodos numéricos tradicionales, cuidadosamente estudiados para poder garantizar su convergencia en un número razonable de iteraciones o pasos. Uno de los más utilizados es el método de diferencias finitas, que se fundamenta en discretizar el dominio de nuestras variables y aplicar un procedimiento iterativo que permita obtener con precisión los valores de la solución.

Esta forma de afrontar este tipo de problemas tiene ciertas limitaciones. La dependencia de un dominio discreto inevitablemente hace que no se disponga de una solución para cualquier punto, teniendo que repetir el proceso en caso de necesitar la solución en un punto externo a la malla. Además, la obtención de una solución exacta suele ser imposible en problemas de alta dimensión o con geometrías irregulares, por lo que la mayoría de las veces es indispensable proceder de manera numérica.

Si se combina todo esto con el auge de los modelos de aprendizaje automático en los últimos años, parece buena idea intentar subsanar estas problemáticas mediante un enfoque nuevo aún no demasiado explorado. Una de las grandes ventajas es la unificación del procedimiento a utilizar para resolver una ecuación, pues en este campo hay una inmensa variedad de casuísticas y cada una de ellas requiere de un análisis previo, generalmente complejo, para garantizar convergencia mediante los métodos tradicionales.

Además, se verá que, al entrenar estos modelos para que se ajusten a la mayor cantidad de puntos de un dominio finito, dispondremos de una solución fiable para cada punto dentro de este. El objetivo será, pues, explorar cómo estas técnicas pueden ser utilizadas y qué resultados se obtienen a partir de ellas para así poder extraer conclusiones sobre qué metodología ofrece un mejor desempeño. Con esto se podrá responder a si, actualmente, es realmente interesante desde algún punto de vista utilizar machine learning frente a los métodos convencionales.

Se aprovecha este pequeño espacio para comentar que este trabajo está fuertemente respaldado por código, del cual es necesario tener disponibilidad para un correcto seguimiento del proyecto. Este puede ser encontrado en [GitHub](#) y para visualizarlo solo se necesita disponer de MATLAB y de un visor de Python, como Visual Studio Code.

Capítulo 2

Teoría Fundamental de Ecuaciones en Derivadas Parciales

2.1 Nociones básicas

En esta sección se pretenden presentar todos los conceptos necesarios para comprender los sistemas de ecuaciones en derivadas parciales que se van a estudiar, de manera que puedan entenderse con facilidad. Las ecuaciones en derivadas parciales consisten, esencialmente, en transcribir un problema a lenguaje matemático de forma que queden bien definidas sus propiedades.

Imagínese que pretende estudiar cómo cambia la temperatura de la superficie de una mesa metálica sobre la que incide el sol desde una dirección concreta, llámese x .

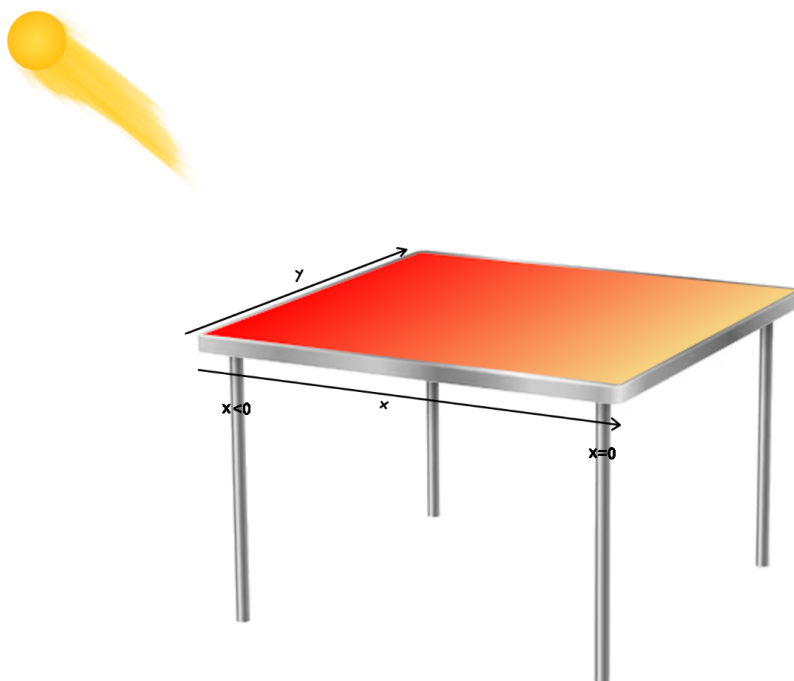


Figura 1: La luz del sol genera una diferencia de temperatura a lo largo de la dirección x .

Se comienza así definiendo una *función* que simbolice el valor de la temperatura, $T(x, y)$. Como se puede apreciar, esta temperatura no cambia en la dirección y , lo cual se simboliza matemáticamente mediante la ecuación

$$\frac{\partial T}{\partial y}(x, y) = 0,$$

de forma que la temperatura solo depende de la dirección x . Si bien en esta dirección no se define bien lo que sucede, está claro que la temperatura cambia a lo largo de ella, de manera que pueden estar involucradas derivadas parciales en x que dificulten el problema de calcular $T(x, y)$.

Por ejemplo, si midiendo la temperatura a lo largo de x se conoce que esta cambia de acuerdo con

$$\frac{\partial T}{\partial x}(x, y) = -1$$

se tendrá, integrando, $T(x, y) = -x + C$, donde C es una constante que marca la temperatura del borde derecho de la mesa, la cual va creciendo linealmente a medida que se avanza hacia el otro borde, más afectado por el sol.

A esta función temperatura le llamaremos *variable dependiente*, pues depende de las *variables independientes* (x, y) de posición, que están definidas dentro de un dominio, en este caso sería la superficie de la mesa.

A lo largo del trabajo se verán casos concretos explicados con mucho más rigor, los cuales serán la *ecuación del calor*, la *ecuación de Poisson* y la *ecuación de Navier-Stokes*.

2.1.1 Definiciones

Formalmente, una **ecuación en derivadas parciales** o **EDP** no es más que una ecuación diferencial en la que, además de estar involucradas las variables independientes y la dependiente, también aparecen derivadas parciales de la variable dependiente con respecto a las independientes. En su forma más general, una EDP puede ser escrita como

$$f(x_1, x_2, \dots, x_n, u, u_{x_1}, u_{x_2}, \dots, u_{x_1 x_1}, u_{x_1 x_2}, \dots) = 0, \quad (2.1)$$

donde u es una función que depende de las variables independientes $x_1, x_2, \dots, x_n$.

Expresaremos las *derivadas parciales* como $u_{x_1}, u_{x_2}, \dots, u_{x_1 x_1}, u_{x_1 x_2}, \dots$, es decir,

$$u_{x_1} = \frac{\partial u}{\partial x_1}, \quad u_{x_1 x_2} = \frac{\partial^2 u}{\partial x_2 \partial x_1}.$$

Además, se definirán los operadores ∇ (*nabla*, *gradiente*, *divergencia*) y Δ (*laplaciano*) como

$$\nabla = \left(\frac{\partial}{\partial x_1}, \frac{\partial}{\partial x_2}, \dots, \frac{\partial}{\partial x_n} \right)^T, \quad \Delta = \nabla^2 = \sum_{i=1}^n \frac{\partial^2}{\partial x_i^2}.$$

Aquí el superíndice T indica la operación de *transposición*. Se indica también que, generalmente, las letras mayúsculas definirán *matrices*, las letras en negrita irán asociadas a *vectores* y las letras sin nada representarán escalares o *campos escalares* (funciones con llegada a $\mathbb{R}$).

A las funciones que satisfacen la ecuación (2.1) en un determinado dominio se les llamará **soluciones**, las cuales forman una familia entera de funciones. De esta forma, las EDPs necesitan de una serie de condiciones para asegurar una solución única, si esta existiese. Se conoce como **orden** de la ecuación diferencial al orden de derivación más alto que aparece en la ecuación, independientemente de las variables independientes involucradas.

Se dirá que una ecuación es **lineal** si es lineal con respecto a u y todas sus derivadas parciales (solo podrán ir elevadas a 1), por lo que los coeficientes serán funciones únicamente de las variables independientes. Este es el caso de la ecuación del calor y de Poisson, mientras que la adaptación de la ecuación de Navier-Stokes que se tratará será no lineal.

La no linealidad puede conducir a comportamientos de solución ricos y complejos, incluidos fenómenos como ondas de choque, **vórtices** y dinámicas caóticas, que normalmente no están presentes en las EDPs lineales.

En el ejemplo de la mesa antes visto pueden existir fuentes adicionales perpetuas de calor. Estas fuentes se describen mediante el término de la ecuación que depende únicamente de las variables independientes. Si es nulo, se dirá que la ecuación es **homogénea**. En caso contrario, será no homogénea.

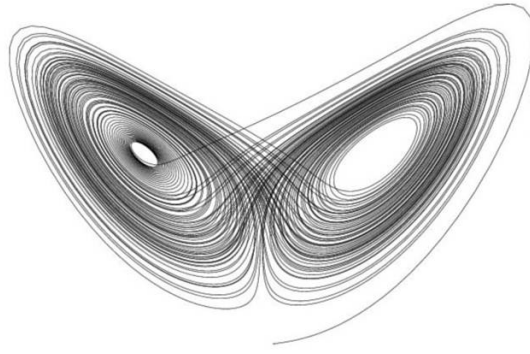


Figura 2: El atractor de Lorentz es el ejemplo clásico de sistema caótico.

2.1.2 Problema de valor inicial

Se recuerda el ejemplo muy sencillo del comienzo de la sección. Allí se dio una función $T(x, y) = x + C$ general que satisfacía la EDP presentada. Esto no es una solución, es una familia entera de ellas, pues la constante C es completamente libre, de forma que debe ser determinada para dar la solución concreta al problema.

De una manera algo más general, se considera una ecuación diferencial parcial de segundo orden para la función u en las variables independientes x_1 y x_2 . Supongamos que esta ecuación se puede resolver explícitamente para $u_{x_2x_2}$, de modo que puede representarse en la forma

$$u_{x_2x_2} = F(x_1, x_2, u, u_{x_1}, u_{x_2}, u_{x_1x_2}, u_{x_1x_2}). \quad (2.2)$$

Para un cierto valor $x_2 = x_2^0$, fijamos los valores iniciales de la función desconocida y de su derivada con respecto a x_2

$$u(x_1, x_2^0) = f(x_1), \quad u_{x_2}(x_1, x_2^0) = g(x_1). \quad (2.3)$$

El problema de determinar la solución de la ecuación (2.2) que satisface las condiciones iniciales (2.3) se conoce como el *problema de valor inicial*. Estas **condiciones iniciales** son necesarias para obtener una solución concreta, fijando las constantes al integrar $u_{x_2x_2}$, análogamente al ejemplo presentado.

A lo largo de este trabajo se utilizarán estas condiciones única y exclusivamente para definir el valor de la solución a tiempo inicial $t = 0$ en las ecuaciones que lo requieran, lo

cual no quiere decir que la condición $T(0, y) = C$ mencionada anteriormente no sea una condición inicial.

2.1.3 Condiciones de contorno

A lo largo del documento, cuando la solución esté confinada en un espacio finito Ω , a los valores que toma la solución a lo largo del borde del dominio se les llamará **condiciones de contorno**. Estas son esenciales para garantizar la unicidad de la solución de una ecuación en un dominio y para modelar correctamente el intercambio físico (como temperatura o flujo) en sus límites.

En el ejemplo mencionado, sería el análogo a definir el valor de la temperatura a lo largo de todo el borde de la superficie de la mesa, quedando definido igualmente de esta forma el valor de C . Estas condiciones proporcionan además los valores necesarios en la frontera para que los métodos numéricos puedan iniciar y converger a una solución única.

Se distinguen tres categorías, claves para el desarrollo de este trabajo. Las condiciones de tipo **Dirichlet** son aquellas en las cuales se conoce exactamente el valor de la solución en la frontera $\partial\Omega$. Son las condiciones más sencillas y directas, siendo utilizadas tanto por la ecuación del calor como por la de Poisson en apartados posteriores.

Estas condiciones pueden ser también de tipo **Neumann**, la cuales definen el valor de la derivada normal de la solución a lo largo del dominio. La derivada normal se define como $\frac{\partial f}{\partial n}(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \mathbf{n}(\mathbf{x})$, $\mathbf{x} \in \partial\Omega$, donde $\mathbf{n}$ es un vector unitario normal en cada punto de la frontera que va en el sentido exterior a esta. En el ejemplo visto, sería como fijar el flujo de temperatura a través de los bordes de la mesa, de forma que al imponer una derivada normal nula o un flujo nulo sobre uno de los lados de la mesa, se estaría indicando que esa parte está térmicamente aislada.

A menudo, en problemas de física, es necesario aplicar condiciones de contorno *mixtas*. Esto significa que la frontera del sistema se divide en conjuntos disjuntos en los cuales se establecen diferentes condiciones de contorno. Este será el caso de la ecuación de Poisson que trataremos, que además de las ya mencionadas condiciones Dirichlet constará de condiciones Neumann.

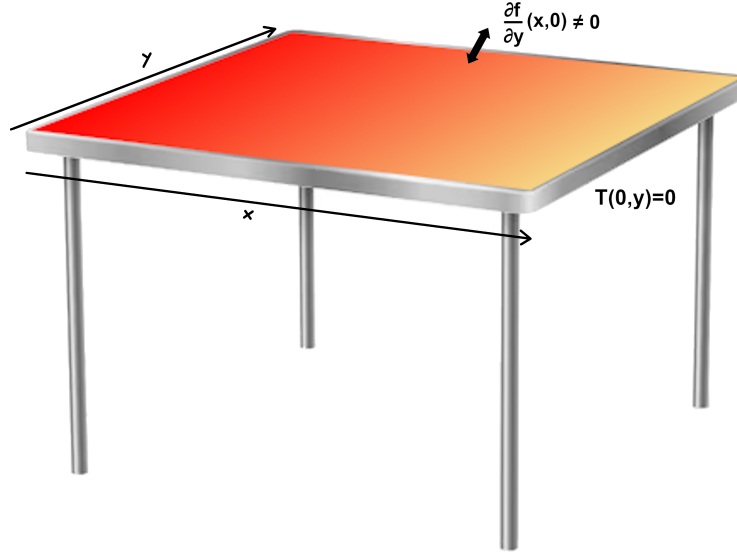


Figura 3: Las condiciones Neumann simbolizan cómo el borde intercambia temperatura con el medio, mientras que las condiciones Dirichlet fijan el valor de la temperatura en la frontera, como si estuviese en contacto con una fuente perpetua de temperatura estable.

En ciertos problemas físicos o de simulación numérica, resulta conveniente imponer que la solución sea **periódica** en una o varias direcciones, de modo que los valores en una cara del dominio coincidan con los de la cara opuesta, simulando un dominio sin bordes. En el ejemplo de la mesa, sería como imponer que las dos fronteras con y constante tengan el mismo valor de temperatura, de forma que si L_y es la longitud de la mesa en esa dirección, $T(x, y) = T(x, y + L_y)$, lo cual de hecho se cumple.

Formalmente, una solución satisface condiciones de contorno periódicas de *periodo* P (anteriormente L_y) en la dirección x_j si, para todo $\mathbf{x} = (x_1, \dots, x_N)$ en el dominio y para cada $i \in \{1, \dots, N\}$, se cumple

$$u(x_1, \dots, x_j + P, \dots, x_d) = u(x_1, \dots, x_j, \dots, x_d).$$

Esta discusión ha sido extraída de [1, capítulo 1], donde se puede encontrar más información sobre los fundamentos y clasificación de las EDPs.

2.2 La ecuación del calor

2.2.1 El problema del calor y su significado

El problema general inicial del que surge esta ecuación es el de modelar cómo evoluciona la temperatura de una determinada región a lo largo del tiempo. Por simplicidad, aquí se hablará siempre de un espacio de una dimensión, $x \in [0, L]$, y un tiempo no negativo $t \geq 0$.

Este es el caso de una varilla delgada de longitud L , de la cual se quiere conocer su temperatura $u(x, t)$ en el punto x y tiempo t . La condición unidimensional se traduce en que la varilla no intercambia calor lateralmente, solo puede hacerlo a lo largo de la dirección x y en principio está en contacto con el exterior a través de sus extremos.

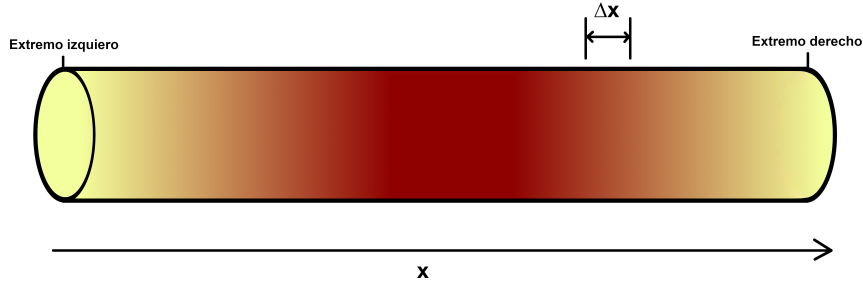


Figura 4: Dibujo de una varilla con zonas a distinta temperatura, representadas por color. Se define el segmento Δx como un desplazamiento infinitesimal en la dirección x .

Se considerará que la varilla es homogénea, lo cual se traduce en que conceptos como la densidad ρ ($\frac{\text{masa}}{\text{longitud}}$), la capacidad calorífica c ($\frac{\text{energía}}{\text{masa} \times \text{temperatura}}$) o la conductividad térmica k ($\frac{\text{energía}}{\text{longitud} \times \text{temperatura} \times \text{tiempo}}$) no cambian a lo largo de x o t .

Está comprobado que el **flujo** $q(x, t)$ de calor entre puntos cercanos depende de la diferencia de temperatura. Así pues, se toma un segmento diminuto de varilla que va de x a $x + \Delta x$. Durante un breve intervalo de tiempo pequeño Δt , la energía interna cambia únicamente por el calor que entra o sale por los extremos. Por tanto [15]

$$\underbrace{\rho c \Delta x u(x, t)}_{\text{energía interna inicial}} + \underbrace{q(x, t) \Delta t}_{\text{calor entrante}} - \underbrace{q(x + \Delta x, t) \Delta t}_{\text{calor saliente}} = \underbrace{\rho c \Delta x u(x, t + \Delta t)}_{\text{energía interna final}}.$$

(I) $\rho c \Delta x u(x, t)$ es la energía almacenada en ese pedacito de varilla al tiempo t .

- $\rho \Delta x$ es la masa de ese segmento (densidad $\times$ longitud).

- Multiplicada por c da la energía necesaria para elevar un grado esa masa.
 - Finalmente, al multiplicar por $u(x, t)$ se obtiene la energía térmica total.
- (II) $q(x, t) \Delta t$ representa el calor que llega por el extremo izquierdo.
- $q(x, t)$ mide “cuánta energía pasa por unidad de longitud y por unidad de tiempo” en x .
 - Al multiplicarlo por Δt se obtiene la energía neta que entra en ese tiempo.
- (III) $-q(x + \Delta x, t) \Delta t$ es el calor que sale por el extremo derecho.
- El signo negativo indica que esa energía abandona el segmento, reduciendo su contenido energético.
- (IV) $\rho c \Delta x u(x, t + \Delta t)$ es la energía interna al final del intervalo.
- Si el calor entrante excede al saliente, esta cantidad será mayor que la inicial, y viceversa.

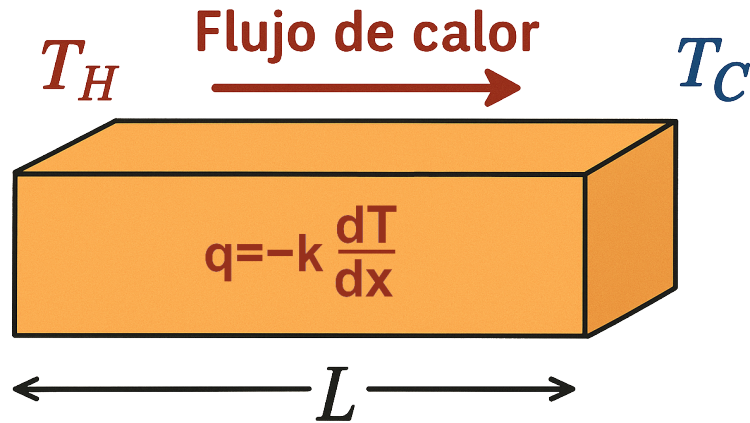


Figura 5: Diagrama esquemático de cómo se comporta el flujo de calor de acuerdo a la Ley de Fourier, con T_H temperatura mayor y T_C la menor.

Si se divide por $\Delta x \Delta t$ y se pasa al límite $\Delta x, \Delta t \rightarrow 0$, por la definición de derivada el término temporal queda como

$$\rho c \frac{u(x, t + \Delta t) - u(x, t)}{\Delta t} \longrightarrow \rho c \frac{\partial u}{\partial t}(x, t),$$

que es la tasa de cambio de la energía interna por unidad de longitud. El término espacial, de igual forma, da

$$\frac{q(x, t) - q(x + \Delta x, t)}{\Delta x} \longrightarrow -\frac{\partial q}{\partial x}(x, t),$$

mide la variación del flujo de calor a lo largo de la barra. Obtenemos así el balance local

$$\rho c u_t = -q_x.$$

Aplicando la ley de Fourier, la cual es una ley física que establece que el flujo de calor entre dos cuerpos es directamente proporcional a la diferencia de temperatura entre ambos,

$$q(x, t) = -k u_x,$$

donde el signo negativo indica que el calor fluye desde zonas más calientes a más frías. Sustituyendo y usando que k es constante, resulta

$$-q_x = -\partial_x(-k u_x) = k u_{xx},$$

y, por tanto,

$$\rho c u_t = k u_{xx}.$$

Dividiendo entre ρc y definiendo el **coeficiente de difusión térmica** $\alpha = \frac{k}{\rho c}$, llegamos a la forma canónica de la ecuación del calor

$$u_t = \alpha u_{xx},$$

que expresa de modo claro la idea de que la velocidad de cambio de temperatura en el tiempo (u_t) es proporcional a la **curvatura** espacial (u_{xx}), modulada por la propiedad del material α . El concepto de curvatura será clave también para el resto de ecuaciones.

Esta ecuación es también conocida como *ecuación de difusión* por la literatura, pues normalmente se plantea el caso de temperatura nula en el medio exterior, de forma que la varilla pierde toda su temperatura tras un tiempo infinito.

2.2.2 Planteamiento formal del problema

Se tratará aquí el caso de la barra homogénea de longitud L descrito anteriormente, el cual debe formalizarse y concretarse para poder ser tratado tanto analíticamente como numéricamente. Para ello se definirán la condición inicial a tiempo cero $f(x)$ y la condición

de contorno en los extremos

$$\left\{ \begin{array}{l} u_t = \alpha u_{xx}, \quad 0 < x < L, \quad t > 0, \\ u(0, t) = 0, \quad t \geq 0, \\ u(L, t) = 0, \quad t \geq 0, \\ u(x, 0) = f(x) = e^{-(x-5)^2}, \quad 0 \leq x \leq L. \end{array} \right. \quad (2.4)$$

Notar que la difusión del calor se realiza a través de los extremos, lo cual queda reflejado mediante las condiciones de contorno Dirichlet, pues la temperatura $u(x, t)$ solo puede ser ≥ 0 y tenderá a fluir hacia los bordes, anulándose.

Para hallar la solución real se seguirá el método de **separación de variables** expuesto por [2, 2.3.2 *Separation of Variables*], de forma que si se asume una solución de la forma

$$u(x, t) = X(x)T(t) \neq 0,$$

se obtiene de la ecuación (2.4), con ' simbolizando la derivada con respecto a la variable correspondiente,

$$XT' = \alpha X''T.$$

Así, se tiene

$$\frac{X''}{X} = \frac{T'}{\alpha T} = -\beta^2,$$

donde β es una constante real, pues según el término de la izquierda o bien depende exclusivamente de X o es una constante y según el central o depende exclusivamente del tiempo o es una constante. Se elige β^2 para ignorar los casos con constantes nulas ($\beta = 0$, da lugar a soluciones triviales) y negativas (β complejo, no da lugar a una base ortogonal útil para representar funciones no triviales). Por lo tanto, X y T deben satisfacer

$$X'' + \beta^2 X = 0, \quad (2.5)$$

$$T' + \beta^2 \alpha T = 0. \quad (2.6)$$

A partir de las condiciones de contorno y de (2.5), se llega al problema de valores propios

$$X'' + \beta^2 X = 0, \quad X(0) = 0, \quad X(L) = 0.$$

Aplicando las condiciones de contorno se tiene la solución

$$X(x) = B \sin(\beta x), \quad \beta = \frac{n\pi}{L}, \text{ para } n = 1, 2, 3, \dots$$

Siguiendo un procedimiento similar con la ecuación (2.6) se obtiene la solución

$$T(t) = C e^{-\beta^2 \alpha t}.$$

Por lo tanto, la solución no trivial de la ecuación del calor que satisface las dos condiciones de contorno es

$$u_n(x, t) = X_n(x)T_n(t) = a_n e^{-(n\pi/L)^2 \alpha t} \sin\left(\frac{n\pi x}{L}\right), \quad n = 1, 2, 3, \dots,$$

donde $a_n = B_n C_n$ es una constante arbitraria.

Por el principio de superposición, que permite sumar todas y cada una de estas soluciones concretas para construir una solución general o en **serie formal**, se obtiene

$$u(x, t) = \sum_{n=1}^{\infty} u_n(x, t) = \sum_{n=1}^{\infty} a_n e^{-(n\pi/L)^2 \alpha t} \sin\left(\frac{n\pi x}{L}\right).$$

Esta solución satisface la condición inicial si

$$u(x, 0) = f(x) = \sum_{n=1}^{\infty} a_n \sin\left(\frac{n\pi x}{L}\right).$$

Esto es cierto si $f(x)$ puede representarse mediante una serie de senos de Fourier con coeficientes de Fourier, lo cual está garantizado por ser de clase C^∞

$$a_n = \frac{2}{L} \int_0^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx.$$

Por lo tanto,

$$u(x, t) = \sum_{n=1}^{\infty} \left[\frac{2}{L} \int_0^L f(\tau) \sin\left(\frac{n\pi\tau}{L}\right) d\tau \right] e^{-(n\pi/L)^2 \alpha t} \sin\left(\frac{n\pi x}{L}\right)$$

es la solución en serie formal del problema de conducción de calor.

Un análisis parecido mucho más completo puede encontrarse en [1, capítulo 7].

2.3 La ecuación de Poisson

2.3.1 El problema Poisson y su significado

A finales del siglo XVII, Newton formuló la *ley de gravitación universal*

$$F = G \frac{m_1 m_2}{r^2},$$

la cual es una aproximación que describe muy bien la *fuerza gravitatoria* entre objetos con alta *densidad de masa*, como podrían ser la Tierra y el Sol. En la ecuación aparece la constante de gravitación universal G , la masa de los objetos m_1 , m_2 y la distancia entre ellos r .

Su escritura en más dimensiones, considerando una densidad de masa cambiante a lo largo de la superficie (como es el caso real de una estrella, que no tiene igual densidad en todo su volumen) e integrales para describir la interacción gravitatoria, llevó a la introducción de una función llamada *potencial gravitatorio* Φ tal que

$$\mathbf{F} = -m\nabla\Phi,$$

donde m es simplemente la masa del objeto (como puede ser la Tierra) sobre el que se quiere conocer qué fuerza gravitatoria está sufriendo (causada por el Sol y otros planetas del sistema solar).

Esta función es mucho más manejable desde el punto de vista matemático y es un campo escalar, es decir, tiene un único valor definido en cada punto del espacio en lugar de un vector, sin perder ningún tipo de información.

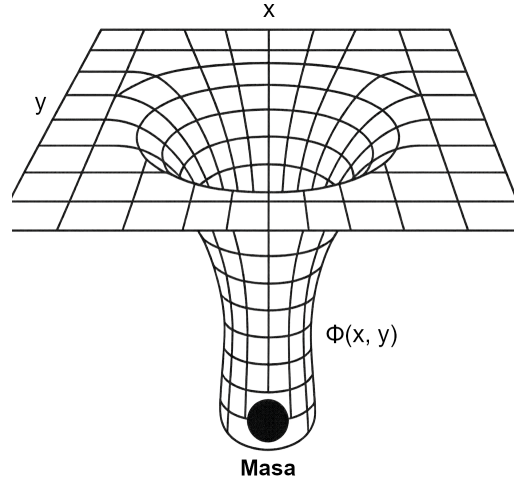


Figura 6: Representación típica del potencial gravitatorio, la curvatura que sufre el espacio por efecto de una masa.

El problema general consiste en modelar el potencial gravitatorio $\Phi(x, y)$ generado por una densidad superficial de masa $\rho(x, y)$ en un dominio plano $\Omega \subset \mathbb{R}^2$. Como ejemplo, tomaremos la densidad superficial en una porción cuadrada del espacio con una estrella en el centro, de forma que en el interior de la estrella, a medida que se avanza hacia su centro, esta densidad crece y en el exterior de esta la densidad vale 0.

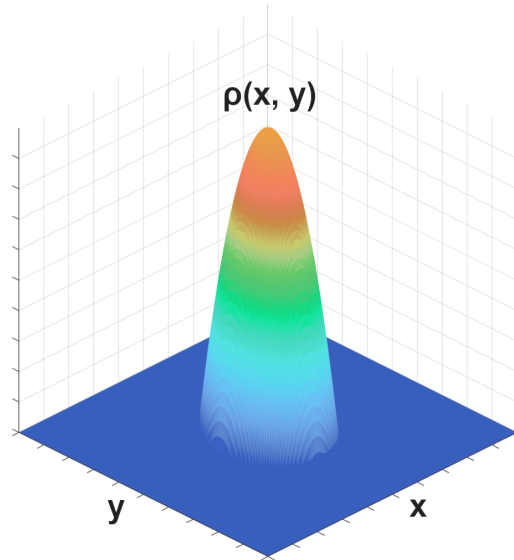


Figura 7: Representación intuitiva de la distribución de masa en una cuadrícula.

Definiendo su contorno por $C = \partial\Omega$, la *ley de Gauss* en dos dimensiones afirma [13]

$$\oint_C \mathbf{F} \cdot \mathbf{n} ds = -2\pi G M_\Omega,$$

donde la integral anterior con un círculo simboliza que se está integrando sobre un contorno cerrado y

$$M_\Omega = \iint_\Omega \rho(x, y) dA \quad \text{es la masa total encerrada.}$$

Entonces

$$\oint_C -\nabla\Phi \cdot \mathbf{n} ds = -2\pi G \iint_\Omega \rho dA.$$

El *teorema de la divergencia* en el plano convierte el flujo de un campo alrededor de C en la integral de su divergencia sobre Ω

$$\oint_C \nabla\Phi \cdot \mathbf{n} ds = \iint_\Omega \nabla \cdot (\nabla\Phi) dA = \iint_\Omega \nabla^2\Phi dA.$$

Por tanto,

$$-\iint_\Omega \nabla^2\Phi dA = -2\pi G \iint_\Omega \rho dA \implies \iint_\Omega \nabla^2\Phi dA = 2\pi G \iint_\Omega \rho dA.$$

Como esto vale para toda región Ω , los integrandos coinciden punto a punto y

$$\nabla^2\Phi(x, y) = 2\pi G \rho(x, y). \implies -\Delta\Phi = f(x, y)$$

2.3.2 Planteamiento formal del problema

Aquí se tratará este mismo caso pero con una función $f(x, y)$ arbitraria, que no necesariamente tiene que simbolizar un problema físico real. Los parámetros de este problema concreto se eligieron de forma que la presentación de los resultados sea visual y cómoda para el lector y que el procedimiento a seguir no sea extremadamente complicado.

Sea pues $\Omega = (a, a + L_x) \times (b, b + L_y)$ un dominio rectangular y $\partial\Omega$ su frontera, a partir de la función continua $f : \Omega \rightarrow \mathbb{R}$ se busca $u : \bar{\Omega} \rightarrow \mathbb{R}$ solución de

$$-\Delta u(x, y) = f(x, y) \quad \forall (x, y) \in \Omega.$$

Concretamente, se trabajará con el problema con condiciones de contorno mixtas

$$\left\{ \begin{array}{ll} f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y), & \forall (x, y) \in \Omega, \\ u(x, y) = 0, & \text{sobre } \Gamma_N = (a, a + L_x) \times \{b + L_y\}, \\ u(x, y) = 0, & \text{sobre } \Gamma_O = \{a\} \times [b, b + L_y], \\ u(x, y) = 0, & \text{sobre } \Gamma_E = \{a + L_x\} \times [b, b + L_y], \\ \frac{\partial u}{\partial n}(x, b) = -\frac{\partial u}{\partial y}(x, b) = 0, & \text{sobre } \Gamma_S = (a, a + L_x) \times \{b\}. \end{array} \right. \quad (2.7)$$

donde $\Gamma_{N,O,E,S}$ son las fronteras norte, oeste, este y sur del rectángulo, respectivamente, y la derivada en la frontera sur se ha calculado de la manera expuesta en la definición.

La función $u(x, y) = \sin(\pi x) \cos(\pi y)$ es solución del sistema anterior con $a = 0$, $b = \frac{1}{2}$, $L_x = 2$ y $L_y = \frac{5}{2}$.

Para formalizar esta ecuación se ha tomado como referencia [3], del cual se utilizará posteriormente su notación y manera de proceder a la hora de analizar la convergencia de los métodos de diferencias finitas.

2.4 Las ecuaciones de Navier-Stokes

2.4.1 Idea intuitiva

Para entender las ecuaciones de Navier-Stokes, se hace necesario presentar primeramente los elementos físicos que la componen, para lo cual se tomará de base [14]. Un **fluido**, como el agua o el aire, es un conjunto de partículas unidas entre sí por fuerzas débiles.

Lo primero que se asumirá es la *ley de la conservación de la masa*, es decir, si se tiene un fluido en una determinada región, la cantidad de masa que entra en esta región debe igualar a la que sale de ella, de modo que no “aparezca” ni “desaparezca” fluido. A esta propiedad se le conoce como **incompresibilidad** y se traduce en que la divergencia del campo de velocidad $\mathbf{v}(x, y, z, t)$ es cero

$$\nabla \cdot \mathbf{v} = 0.$$

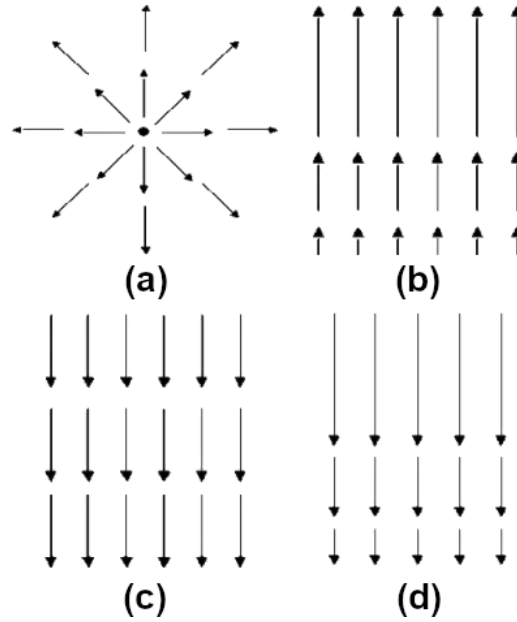


Figura 8: Los campos mostrados arriba tienen todos divergencia no nula salvo (c), en el que se puede apreciar que sea cual sea la región que se defina siempre entrará la misma cantidad de fluido que la que sale.

Para describir cómo cambia la velocidad del fluido, se utiliza el concepto de *inercia*. La inercia se define intuitivamente como la “resistencia” al cambio de movimiento que experimenta el fluido. Este concepto se describe por convenio en dinámica de fluidos mediante la **derivada material o convectiva**, que se escribe como

$$\frac{D\mathbf{v}}{Dt} = \underbrace{\frac{\partial \mathbf{v}}{\partial t}}_{\text{cambio local en un punto}} + \underbrace{(\mathbf{v} \cdot \nabla)\mathbf{v}}_{\text{cambio por movimiento}}$$

mide cómo varía la velocidad $\mathbf{v}$ que “siente” realmente una partícula a medida que se desplaza.

El término $\frac{\partial \mathbf{v}}{\partial t}$ mide cómo cambia la velocidad por el simple paso del tiempo, lo que se puede considerar el concepto clásico de aceleración de una partícula. El término $(\mathbf{v} \cdot \nabla)\mathbf{v}$ es el que provoca que esta ecuación se incruste dentro de las ecuaciones no lineales, a diferencia de las dos anteriores, y simboliza qué parte del cambio de velocidad se debe a que nos movemos hacia zonas donde el fluido va más rápido o más lento.

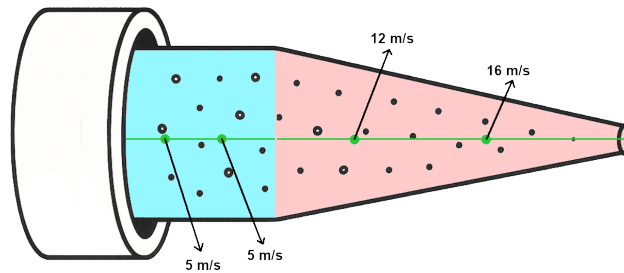


Figura 9: La manera más simple de visualizar este concepto es mediante una manguera. A lo largo del cilindro, el agua mantiene la misma velocidad debido a que la forma de la manguera no cambia. Sin embargo, al llegar a la boquilla, el cambio abrupto en la forma produce que las partículas se compriman más y experimenten una aceleración.

Se precisa conocer también el concepto de presión p , la cual es la fuerza que ejerce el fluido por unidad de área sobre cualquier superficie imaginaria en el punto $\mathbf{x}$. Cuando decimos que el fluido “tiene más presión” en un sitio, significa que allí empujaría con más fuerza sobre una pared o sobre otro cúmulo de fluido.

Así pues, el término $-\nabla p$ simboliza el movimiento que se produce debido a la presión interna del fluido. En cada punto, la presión p varía en el espacio y tiende a empujar el fluido desde las regiones donde p es mayor hacia aquellas donde p es menor. Dicho de otra manera, una partícula del flujo tiende a alejarse de zonas muy comprimidas, yendo de manera natural hacia una zona de menor compresión.

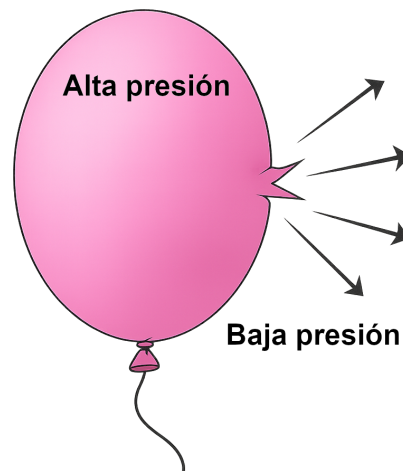


Figura 10: La presión interior del globo es mayor que la presión atmosférica del aire que le rodea. Esta fuerza hace que se mantenga la del globo siempre y cuando el material la aguante, pero cuando se abre un agujero en algún lugar de la superficie el aire escapa debido a la diferencia de presión.

Por último, el término $\Delta \mathbf{v}$ mide cuán bruscos son los cambios de velocidad en función del entorno, análogo a la ecuación de Poisson, pues es el mismo concepto de reacción de una masa a la curvatura del espacio pero en este caso con velocidades. Suele ir multiplicado por lo que se conoce como **viscosidad** cinemática ν , la cual difumina esas diferencias haciendo que las regiones con velocidades muy distintas se “suavicen” gradualmente.

Juntando todo esto, las **ecuaciones de Navier-Stokes** toman la forma de

$$\begin{cases} \frac{D\mathbf{v}}{Dt}(\mathbf{x}, t) = -\nabla p(\mathbf{x}, t) + \nu \Delta \mathbf{v}(\mathbf{x}, t), \\ \operatorname{div} \mathbf{v}(\mathbf{x}, t) = 0, \\ \mathbf{v}(\mathbf{x}, 0) = \mathbf{v}_0(\mathbf{x}), \end{cases} \quad \begin{aligned} &(\mathbf{x}, t) \in \mathbb{R}^d \times [0, \infty), \\ &\mathbf{x} \in \mathbb{R}^d, \end{aligned}$$

en dimensión $d = 2$ o 3 , donde el último término no es más que la condición inicial.

2.4.2 Particularización del problema

A continuación, se desarrollará una variación más simple de esta ecuación desde el punto de vista numérico, llamada *ecuación de vorticidad*, la cual necesita ciertos recursos previos y, al igual que las ecuaciones de Navier-Stokes, es no lineal.

El primero es que la matriz V de dimensión 3×3 y componentes $(\nabla \mathbf{v})_{ij} = \partial v_i / \partial x_j$ se descompone en su parte simétrica y antisimétrica como

$$\mathcal{D} = \frac{1}{2} \left(\nabla \mathbf{v} + (\nabla \mathbf{v})^T \right),$$

$$\mathcal{R} = \frac{1}{2} \left(\nabla \mathbf{v} - (\nabla \mathbf{v})^T \right).$$

Aquí, $\mathcal{D}$ se denomina matriz de *deformación* y $\mathcal{R}$ matriz de *rotación*.

El segundo es que la **vorticidad** $\boldsymbol{\omega}$ se define como

$$\boldsymbol{\omega} = \nabla \times \mathbf{v} = \left(\frac{\partial v^3}{\partial x_2} - \frac{\partial v^2}{\partial x_3}, \frac{\partial v^1}{\partial x_3} - \frac{\partial v^3}{\partial x_1}, \frac{\partial v^2}{\partial x_1} - \frac{\partial v^1}{\partial x_2} \right)^T$$

y satisface la relación

$$\mathcal{R} \mathbf{h} = \frac{1}{2} \boldsymbol{\omega} \times \mathbf{h}, \quad \forall \mathbf{h} \in \mathbb{R}^3.$$

La vorticidad cuantifica cómo un fluido en un punto dado tiende a girar sobre sí mismo, ofreciendo una medida local de la velocidad angular del mismo. Tiene especial influencia en aerodinámica, meteorología e ingeniería química.

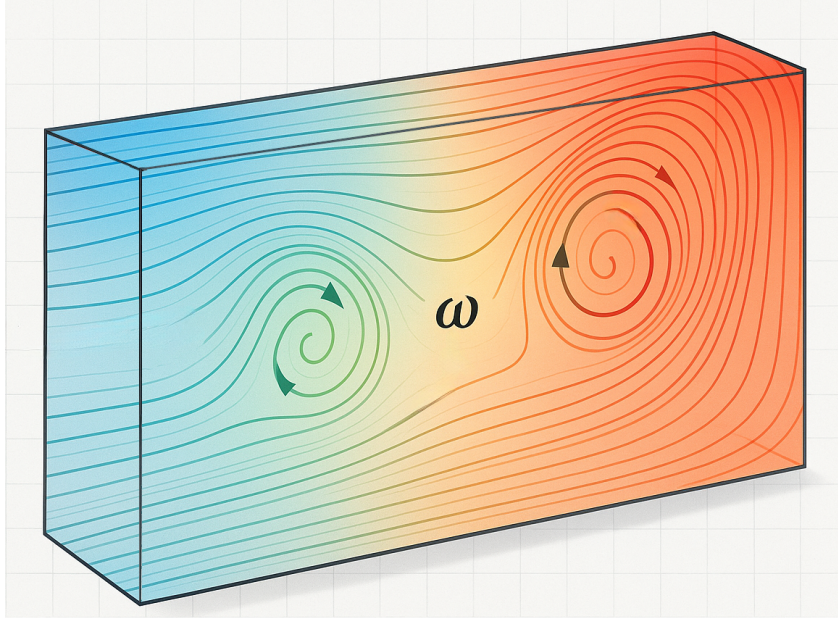


Figura 11: Representación de un fluido con dos vórtices. Las líneas de corriente o streamlines (flechas) muestran la dirección y velocidad del fluido, mientras el color de azul a rojo marca el sentido de rotación local. Notar que el eje de rotación es perpendicular a la superficie en la que se manifiesta el vórtice, pese a poder ser un fluido 2D.

Tras haber definido estas dos herramientas, el siguiente paso es calcular la derivada con respecto a x_k de la ecuación de Navier-Stokes, obteniendo componente a componente

$$\frac{\partial}{\partial t}(v_{x_k}^i) + v^j \frac{\partial}{\partial x_j}(v_{x_k}^i) + \frac{\partial v^j}{\partial x_k} \frac{\partial v^i}{\partial x_j} = -\frac{\partial^2 p}{\partial x_i \partial x_k} + \nu \frac{\partial^2}{\partial x_j \partial x_j}(v_{x_k}^i).$$

Introduciendo la notación

$$V \equiv (v_{x_k}^i) = \left(\frac{\partial v^i}{\partial x_k} \right), \quad P \equiv (p_{x_i x_k}) = \left(\frac{\partial^2 p}{\partial x_i \partial x_k} \right),$$

para la *matriz hessiana* de la presión p . Obtenemos así la ecuación matricial para V

$$\frac{DV}{Dt} + V^2 = -P + \nu \Delta V.$$

A partir de las matrices definidas antes, es natural descomponer

$$V = \underbrace{\frac{1}{2}(V + V^T)}_{\mathcal{D}} + \underbrace{\frac{1}{2}(V - V^T)}_{\mathcal{R}}.$$

de forma que V satisface

$$V^2 = (\mathcal{R}^2 + \mathcal{D}^2) + (\mathcal{D}\mathcal{R} + \mathcal{R}\mathcal{D}),$$

donde el primer término es simétrico y el segundo antisimétrico.

La parte simétrica queda como

$$\frac{D\mathcal{D}}{Dt} + \mathcal{D}^2 + \mathcal{R}^2 = -P + \nu\Delta\mathcal{D},$$

mientras que la parte antisimétrica es

$$\frac{D\mathcal{R}}{Dt} + \mathcal{D}\mathcal{R} + \mathcal{R}\mathcal{D} = \nu\Delta\mathcal{R}.$$

Como $\mathcal{R}\mathbf{h} = \frac{1}{2}\boldsymbol{\omega} \times \mathbf{h}$, tras algunos cálculos directos obtenemos la ecuación de vorticidad

$$\frac{D\boldsymbol{\omega}}{Dt} = \mathcal{D}\boldsymbol{\omega} + \nu\Delta\boldsymbol{\omega}. \quad (2.8)$$

Así pues, se estudiará numéricamente el caso concreto del **vórtice de Taylor–Green**, el cual es un flujo definido en geometría periódica por el campo de velocidad tridimensional

$$\mathbf{v}^x = \sin x \cos y \cos z, \quad \mathbf{v}^y = -\cos x \sin y \cos z, \quad \mathbf{v}^z = 0.$$

Este flujo es un ejemplo prototípico para validación de códigos CFD (dinámica de fluidos computacional) actuales y tuvo cierto interés cuando se propuso debido a que se creía que podría desarrollar singularidades (puntos en los que la solución o alguna de sus derivadas se vuelven infinitas o dejan de estar bien definidas) en tiempo finito.

Se planteará a continuación una variante bidimensional, de forma que se tratará por comodidad $\mathbf{v} = (v^x, v^y, 0) \rightarrow \mathbf{v} = (u, v)$ y $\boldsymbol{\omega} = (0, 0, \omega) \rightarrow \omega = \omega = v_x - u_y = \Delta\psi$, donde ψ se conoce como *streamfunction* y cumple

$$\psi_x = v, \quad \psi_y = -u.$$

Las curvas de nivel de la streamfunction son lo que se conocen como **streamlines** o líneas de flujo, extremadamente útiles para visualizar cómo evoluciona o se mueve un fluido bidimensional y que podrán observarse en el Capítulo 5 de resultados.

Como dominio se tomará $(x, y) \in [0, 2\pi] \times [0, 2\pi]$, $t \geq 0$ y el problema quedará sujeto a las condiciones iniciales y de contorno

$$\begin{cases} \omega(x, y, 0) = -2 \cos x \cos y, \\ \omega(0, y, t) = \omega(2\pi, y, t), \\ \omega(x, 0, t) = \omega(x, 2\pi, t). \end{cases} \quad (2.9)$$

elegido todo lo anterior cuidadosamente para que se pueda apreciar el carácter periódico de la solución con comodidad.

Nótese que en la ecuación (2.8), considerando aún las magnitudes $\mathbf{v}$ y $\boldsymbol{\omega}$ en su forma tridimensional (que realmente nunca dejan de serlo, simplemente han sido reformuladas para que su manejo sea más cómodo), el término $\mathcal{D}\boldsymbol{\omega}$ se anula y se obtiene la ecuación

$$\omega_t + u \omega_x + v \omega_y = \nu(\omega_{xx} + \omega_{yy}). \quad (2.10)$$

Se define la solución de prueba $\omega(x, y, t) = -2 \cos x \cos y e^{-2\nu t}$ y, a partir de (2.10),

$$\begin{aligned} 4\nu\omega + (2u \sin x \cos y + 2v \cos x \sin y)e^{-2\nu t} &= \nu(2\omega + 2\omega) \\ u \sin x \cos y + v \cos x \sin y &= 0. \end{aligned} \quad (2.11)$$

Por tanto, se debe comprobar que se cumple esta ecuación con

$$v = \psi_x = -\sin x \cos y e^{-2\nu t}, \quad u = -\psi_y = \cos x \sin y e^{-2\nu t}.$$

y se puede ver fácilmente que $\psi = \cos x \cos y e^{-2\nu t}$ cumple la ecuación de Poisson antes mostrada. Sustituyendo en (2.11) tenemos el resultado.

Para la teoría básica de este apartado se ha utilizado el capítulo 1 de [4] y, para la adaptación a flujos bidimensionales, el capítulo 2.

Capítulo 3

Formalización del Método de Diferencias Finitas

3.1 Definiciones y resultados clave

Si hay un resultado relevante para construir los **métodos en diferencias finitas** o MDF, ese es el del desarrollo de una función en **serie de Taylor**, el cual establece que, sea una función $f(x): \mathbb{R} \rightarrow \mathbb{R}$ de clase C^n en un entorno de x_0 , puede ser aproximada por

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n + \mathcal{O}((x - x_0)^{n+1}),$$

donde $f^{(n)}$ representa la derivada n -ésima y $\mathcal{O}((x - x_0)^n)$ significa que existe una constante $C > 0$ tal que el error cometido por la aproximación, es decir, la diferencia absoluta entre el valor real y el aproximado, está acotado superiormente por $C|x - x_0|^n$ para todo $x \in \mathbb{R}$.

A la hora de desarrollar los métodos se utilizará el término **mall**a, la cual será la discretización del dominio en el que queramos obtener la solución en puntos o **nod**os. Se tratará con mallas determinadas unívocamente por un número N que representa el tamaño (mayor N , mayor número de nodos, malla más amplia).

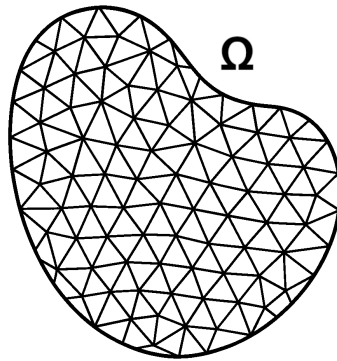


Figura 12: Ejemplo visual de mallado sobre un dominio arbitrario Ω .

Se fija la notación del *operador diferencial* $\mathcal{L}_N$ que, aplicado a una función cualquiera, toma la forma del método de diferencias finitas para cada nodo de la malla que se haya definido, esencialmente evaluando esta función en los nodos correspondientes. Se verán ejemplos concretos que dejarán este concepto más claro en los apartados inmediatamente posteriores.

Se define ahora el **error de consistencia** como un vector

$$r_j = (\mathcal{L}_N u)(x_j) - f(x_j), \quad j = 1, \dots, N,$$

donde $f(x_j)$ es el término no homogéneo. Se debe prestar especial atención a la diferencia de notación para evaluar la solución exacta en un punto $u(x_j)$ y su aproximación por el método u_j .

Así pues, se dice que el esquema de diferencias finitas

$$(\mathcal{L}_N u)_j = f(x_j) \quad j = 1, \dots, N,$$

es *consistente de orden n* con la ecuación diferencial si existen constantes $C > 0$ y $h_0 > 0$ tales que

$$\max_{j=1, \dots, N} |r_j| = \|\mathbf{r}\|_\infty \leq Ch^n, \quad \forall 0 < h < h_0,$$

donde h es el espaciado uniforme de la partición espacial, típicamente $\frac{1}{N}$, y $\|\cdot\|_\infty$ la norma del supremo.

Se define ahora el **error discreto** por $e^j = u(x_j) - u_j$, $j = 1, \dots, N$, de forma que por linealidad

$$(\mathcal{L}_N e)_j = (\mathcal{L}_N u)(x_j) - (\mathcal{L}_N u)_j = (\mathcal{L}_N u)(x_j) - f(x_j) = r_j.$$

Por tanto, se dirá que el esquema de diferencias finitas es *convergente de orden n* con respecto a la *norma* $\|\cdot\|$ genérica bien definida si existen constantes $C > 0$ y $h_0 > 0$ tales que, para todo $0 < h < h_0$,

$$\|\mathbf{e}\| \leq Ch^n.$$

Se presenta el concepto de **estabilidad**, el cual viene a decir intuitivamente que, si se tiene controlado a tiempo inicial el error, a medida que el método avanza hacia nuevos tiempos el error sigue estando controlado. Esto se traduce matemáticamente a que, para

un tiempo T sobre el que se ha definido una malla uniforme de longitud τ , exista una constante $C > 0$, independiente de h , τ y n , de forma que para la norma $\|\cdot\|$ y para todo $n\tau \leq T$ se verifique

$$\|e^n\| \leq C(\|e^0\| + \max_{1 \leq k \leq n} \|r^k\|).$$

Todas estas definiciones se basan en [3, capítulo 2], donde aparece bastante más información y ejemplos.

Con todo esto definido, solo queda comentar el **teorema de Lax–Richtmyer**, crucial en el tratamiento de errores, el cual establece que si un esquema de diferencias finitas es consistente de orden n y estable, entonces es convergente de orden n [3, página 328].

Si bien se harán ciertos comentarios, en este trabajo no se tratarán con excesivo detalle los conceptos de *unicidad* de solución y de *estabilidad* del método, pues definir el proceso con todo lujo de detalle puede ser pesado para el lector y no aporta directamente a lo que es la implementación del método. De tener un gran interés y querer profundizar en lo que se hace, consultar [3, 6.2.2 *Uniqueness and Stability*] puede resultar de gran ayuda.

3.2 Particularización y tratamiento de errores

En esta sección, se presentarán tanto la construcción del esquema de diferencias finitas como el procedimiento necesario para llegar a acotar el error de convergencia para los casos particulares de la ecuación del calor y de Poisson.

La estructura aquí seguida, la cual es la manera natural de proceder, es la de aplicar el teorema de Taylor a cada término de la ecuación para obtener poder reescribirla en función de los nodos de la malla. Seguidamente, para poder resolverla informáticamente, se buscará una manera de escribir el método en formato matricial, de manera que se forme un sistema lineal simple y fácil de resolver para obtener la aproximación.

Por último, se comentará un artículo científico que trata las ecuaciones de Navier-Stokes en general, el cual se ha adaptado y trasladado a **MATLAB** personalmente, para definir un método de diferencias finitas en **MATLAB** y poder compararlo a nivel de rendimiento con la solución de la red neuronal correspondiente.

3.2.1 Ecuación del calor

Para tratar numéricamente la ecuación (2.4), se presenta un desarrollo basado en el esquema de Crank-Nicolson, un método que posee una parte explícita y una implícita idénticamente ponderadas para conseguir mayor orden de convergencia [3, capítulo 4].

Primero se ha de comentar que, si bien la ecuación no presenta un término no homogéneo, el cual es el que solo depende de las variables independientes. Con el objetivo de no alterar demasiado el procedimiento general y visualizar mejor algunos pasos, se definirá $g(x, t) = 0$ como término no homogéneo y se añadirá a la ecuación

$$u_t = \alpha u_{xx} + g(x, t).$$

La malla espacial se define por $x_j = jh$ para $j = 0, 1, \dots, N + 1$, donde $h = \frac{L}{N+1}$, y la malla temporal por $t_m = m\tau$ para $m = 0, 1, \dots, M$, donde $\tau = \frac{T}{M}$.

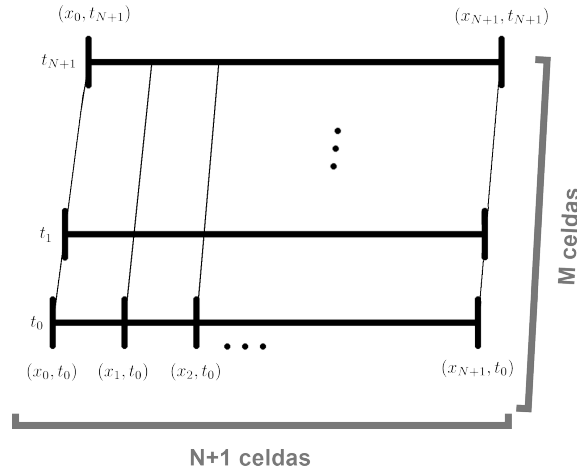


Figura 13: Mallado espacio-temporal para el problema del calor.

A partir de aquí, se tomará $j = 1, \dots, N$ hasta que se diga lo contrario, pues por las condiciones de contorno se conocen exactamente

$$u_0^m = \alpha_0(t_m) = 0, \quad u_{N+1}^m = \beta_0(t_m) = 0 \quad \forall m.$$

Se utilizan los desarrollos de Taylor

$$u(x_j, t_{m+1}) = u(x_j, t_m) + \tau u_t(x_j, t_m) + \frac{\tau^2}{2} u_{tt}(x_j, t_m) + \frac{\tau^3}{6} u_{ttt}(x_j, t_m) + \dots,$$

$$u(x_{j\pm 1}, t_m) = u(x_j, t_m) \pm hu_x(x_j, t_m) + \frac{h^2}{2}u_{xx}(x_j, t_m) \pm \frac{h^3}{6}u_{xxx}(x_j, t_m) + \dots$$

Al sumar los desarrollos espaciales, se obtiene la aproximación

$$u_{xx}(x_j, t_m) \approx \Delta_h u_j^m = \frac{u_{j-1}^m - 2u_j^m + u_{j+1}^m}{h^2},$$

y, para los tiempos, se tiene

$$u_t(x_j, t_m) \approx \frac{u_j^{m+1} - u_j^m}{\tau}.$$

Así se construye el esquema de Crank-Nicolson

$$\frac{u_j^{m+1} - u_j^m}{\tau} = \alpha \frac{\Delta_h u_j^m + \Delta_h u_j^{m+1}}{2} + \frac{g(x_j, t_m) + g(x_j, t_{m+1})}{2},$$

de forma que se puede apreciar una parte **explícita**, que utiliza únicamente valores en el nivel temporal m , y una parte **implícita**, que involucra valores en el nivel $m + 1$, combinando así la sencillez computacional del método explícito con la estabilidad del método implícito.

Sea ahora el operador diferencial mencionado antes

$$(\mathcal{L}_N u)(x_j, t_{m+1}) = \frac{u(x_j, t_{m+1}) - u(x_j, t_m)}{\tau} - \alpha \frac{\Delta_h u(x_j, t_m) + \Delta_h u(x_j, t_{m+1})}{2},$$

el error de consistencia no es más que

$$r_j^{m+1} = (\mathcal{L}_N u)(x_j, t_{m+1}) - \frac{g(x_j, t_m) + g(x_j, t_{m+1})}{2}. \quad (3.1)$$

De lo escrito anteriormente se tiene directamente en t_m

$$\Delta_h u(x_j, t_m) = u_{xx}(x_j, t_m) + \frac{h^2}{12}u_{xxx}(x_j, t_m) + \mathcal{O}(h^4),$$

y análogamente en t_{m+1} ,

$$\Delta_h u(x_j, t_{m+1}) = u_{xx}(x_j, t_m) + \tau u_{xt}(x_j, t_m) + \frac{h^2}{12}u_{xxx}(x_j, t_m) + \mathcal{O}(\tau^2 + h^4).$$

Sustituyendo en r_j^{m+1} y usando la ecuación del calor y su derivada temporal

$$u_t = \alpha u_{xx} + g \longrightarrow u_{tt} = \alpha u_{xxt} + g_t,$$

todos los términos de orden τ se cancelan y queda

$$r_j^{m+1} = -\frac{\alpha h^2}{12} u_{xxxx}(x_j, t_m) + \mathcal{O}(\tau^2 + h^4).$$

Por tanto el esquema de Crank-Nicolson es consistente de orden dos en espacio y tiempo

$$\max_{j=1, \dots, N} |r_j^{m+1}| = \mathcal{O}(\tau^2 + h^2) \quad \forall m = 0, \dots, M.$$

Por comodidad, se comentará la estabilidad después de las formulaciones matriciales del método.

Implementación

Se comenta ahora la transcripción del método a código, comenzando por construir el vector de nodos espaciales

$$\mathbf{x} = [x_0, x_1, x_2, \dots, x_N, x_{N+1}], \quad \text{con } x_0 = 0, \quad x_j = 0 + jh, \quad x_{N+1} = L.$$

Dado que en x_0 y x_{N+1} se imponen condiciones de Dirichlet $u(0, t) = \alpha(t)$ y $u(L, t) = \beta(t)$, sólo se consideran las incógnitas en los nodos interiores

$$\mathbf{x}_{\text{int}} = [x_1, x_2, \dots, x_N],$$

y se denota el vector de valores iniciales en esos nodos como

$$\mathbf{u}_{\text{int}}^0 = [u^0(x_1), u^0(x_2), \dots, u^0(x_N)]^\top.$$

Separar los nodos interiores de los extremos permite trabajar con matrices de tamaño $N \times N$. De este modo, las dos condiciones de contorno en x_0 y x_{N+1} aparecerán como términos adicionales en el vector del término no homogéneo, en lugar de introducir filas rígidas para la frontera.

Para discretizar la derivada segunda espacial, se define la matriz $D \in \mathbb{R}^{N \times N}$ tridiagonal

$$D = \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -1 & 2 & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{pmatrix}.$$

Para construirla introducimos I matriz identidad de tamaño $N \times N$ y E matriz con unos en la superdiagonal y ceros en el resto, de forma que $D = 2I - E - E^\top$. Para generarlas usamos `scipy.sparse`, de manera que en memoria solo se alojen las celdas de la matriz que están siendo utilizadas, sobrentendiéndose que las que no son ceros por el código.

De esta forma, en notación matricial el método antes descrito quedaría como un sistema lineal de la forma

$$A \mathbf{u}_{\text{int}}^{n+1} = B \mathbf{u}_{\text{int}}^n + \frac{\tau}{2}(\mathbf{g}^n + \mathbf{g}^{n+1}) \quad (3.2)$$

para cada tiempo, con

$$A = I + \frac{1}{2}\alpha\lambda D, \quad B = I - \frac{1}{2}\alpha\lambda D, \quad \text{con } \lambda = \frac{\tau}{h^2}.$$

Pero aún faltan por incluir las condiciones de contorno, de forma que añadiendo para cada paso de tiempo los extremos $u(0, t_n) = \alpha(t_n)$ y $u(L, t_n) = \beta(t_n)$ al vector del término no homogéneo, como se mencionó antes, se llega a

$$\mathbf{b}^n = \tau \mathbf{g}(\mathbf{x}_{\text{int}}, t_n) + \lambda \begin{pmatrix} \alpha(t_n) \\ 0 \\ \vdots \\ 0 \\ \beta(t_n) \end{pmatrix} \in \mathbb{R}^N,$$

de forma que el sistema completo a resolver es

$$A \mathbf{u}_{\text{int}}^{n+1} = B \mathbf{u}_{\text{int}}^n + \mathbf{b}^n + \mathbf{b}^{n+1}.$$

Con esto se logran trasladar al lado derecho los términos que involucran u_0 y u_{N+1} en la discretización de la segunda derivada en los índices $j = 1$ y $j = N$, respectivamente, de forma que no se necesita añadir filas dedicadas a ellos en A para la frontera.

En resumen, durante este procedimiento

(I) Se calcula $t_{n+1} = (n + 1)\tau$.

(II) Se construye el vector

$$\mathbf{b}^{n+1} = \tau \mathbf{g}(\mathbf{x}_{\text{int}}, t_{n+1}) + \lambda \begin{pmatrix} \alpha(t_{n+1}) \\ 0 \\ \vdots \\ 0 \\ \beta(t_{n+1}) \end{pmatrix}$$

conocido por las condiciones iniciales y de contorno.

(III) Se calcula el término derecho

$$B \mathbf{u}_{\text{int}}^n + \mathbf{b}^n + \mathbf{b}^{n+1}.$$

(IV) Se resuelve el sistema lineal

$$A \mathbf{u}_{\text{int}}^{n+1} = B \mathbf{u}_{\text{int}}^n + \mathbf{b}^n + \mathbf{b}^{n+1}$$

con `spsolve`, obteniendo $\mathbf{u}_{\text{int}}^{n+1}$ y almacenándolo para la siguiente iteración.

Estabilidad

Se comenzará presentando unos conceptos muy simples sobre normas y matrices. Sea pues $\|\cdot\|$ la **norma euclídea** definida como

$$\|\mathbf{x}\|_2 = (\mathbf{x}^\top \mathbf{x})^{1/2} \quad \mathbf{x} \in \mathbb{R}^N.$$

Se dirá que $\lambda(A)$ es autovalor de $A \in \mathbb{R}^{N \times N}$ si existe $\mathbf{v} \in \mathbb{R}^N$ tal que

$$A\mathbf{v} = \lambda\mathbf{v}.$$

Se define la *norma inducida* sobre matrices en términos de autovalores por

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^\top A)} = \max_k |\lambda_k(A)| \quad A \in \mathbb{R}^{N \times N}$$

donde la segunda igualdad se tiene si A es simétrica y real, $A^\top A = A^2$.

Este tipo de normas tienen la propiedad para $A \in \mathbb{R}^{n \times m}$, $B \in \mathbb{R}^{m \times p}$, de

$$\|AB\|_2 \leq \|A\|_2 \|B\|_2.$$

Dicho esto, se simplifica $g(x, t) = 0$, ya que no aporta nada en este apartado. En estas condiciones, por (3.2)

$$A \mathbf{u}^{n+1} = B \mathbf{u}^n,$$

donde

$$A = I + \frac{\alpha\tau}{2h^2} D, \quad B = I - \frac{\alpha\tau}{2h^2} D,$$

y D es la matriz tridiagonal que aproxima la segunda derivada espacial.

Se define la matriz de evolución

$$G = A^{-1}B,$$

que permite estudiar de manera global y unificada cómo se propagan y amplifican los errores en cada paso temporal, pues según la ecuación (3.1) en forma matricial

$$\mathbf{e}^{n+1} = G\mathbf{e}^n - \tau A^{-1}\mathbf{r}^{n+1}. \quad (3.3)$$

La matriz D es simétrica real y diagonalizable ortogonalmente, con autovalores

$$\mu_k = 4 \sin^2\left(\frac{k\pi}{2(N+1)}\right), \quad k = 1, \dots, N, \quad 0 < \mu_k \leq 4.$$

Como A y B son polinomios de D , se pueden escribir directamente los autovalores de A^{-1}

$$\lambda_k = \frac{1}{1 + \frac{\alpha\tau}{2h^2} \mu_k} \leq \frac{1}{1} = 1,$$

donde se aprecia que, aunque cada λ_k depende de τ/h^2 , la cota no depende de este cociente por ser ≥ 1 .

De manera análoga se pueden definir los autovalores de G

$$\rho_k = \frac{1 - \frac{\alpha\tau}{2h^2}\mu_k}{1 + \frac{\alpha\tau}{2h^2}\mu_k}$$

donde $0 \leq |\rho_k| \leq 1$, por lo que el esquema homogéneo cumple

$$\|G\|_2 = \max_k |\rho_k| \leq 1$$

Iterando en (3.3) desde $\mathbf{e}^0 = \mathbf{0}$,

$$\mathbf{e}^n = G^n \mathbf{e}^0 - \sum_{k=1}^n \tau G^{n-k} A^{-1} \mathbf{r}^k \longrightarrow \|\mathbf{e}^n\| = \left\| \sum_{k=1}^n G^{n-k} A^{-1} \mathbf{r}^k \right\|.$$

por lo que si se consigue acotar el término de la derecha se tendría el resultado.

Utilizando las propiedades de la norma anteriormente vistas y sumando hasta $M = \frac{T}{\tau}$

$$\left\| \sum_{k=1}^n \tau G^{n-k} A^{-1} \mathbf{r}^k \right\| \leq \tau \sum_{k=1}^n \|G^{n-k}\| \|A^{-1}\| \|\mathbf{r}^k\| \leq T (h^2 + \tau^2) \leq C$$

se llega a que el esquema de Crank–Nicolson es estable y convergente de orden dos en espacio y tiempo [3, capítulo 4].

Directamente se aprecia que esto implica la convergencia también del *error cuadrático medio* (**MSE**), definido como

$$\text{MSE} = \frac{1}{N+2} \frac{1}{M+1} \sum_{i=0}^{N+1} \sum_{j=0}^M (u(x_i^j) - u_i^j)^2.$$

3.2.2 Ecuación de Poisson

Se comenzará fijando la notación de la malla espacial bidimensional de acuerdo a la ecuación (2.7). Así pues, sean $N \in \mathbb{N}$, $h_x = \frac{L_x}{N+1}$, $h_y = \frac{L_y}{N+1}$, $x_i = ih_x$ para $i = 0, \dots, N+1$ e $y_j = jh_y$ para $j = 0, \dots, N+1$. Se introduce la notación

$$\bar{\Omega}_N := \{(i, j), \quad 0 \leq i, j \leq N+1\}, \quad \Omega_N := \{(i, j), \quad 1 \leq i, j \leq N\}, \quad \partial\Omega_N = \bar{\Omega}_N \setminus \Omega_N,$$

$$\Gamma_N^N := [1, N] \times \{N+1\}, \quad \Gamma_N^E := \{N+1\} \times [0, N+1], \quad \Gamma_N^S := [1, N] \times \{0\},$$

$$\Gamma_N^O := \{0\} \times [0, N+1].$$

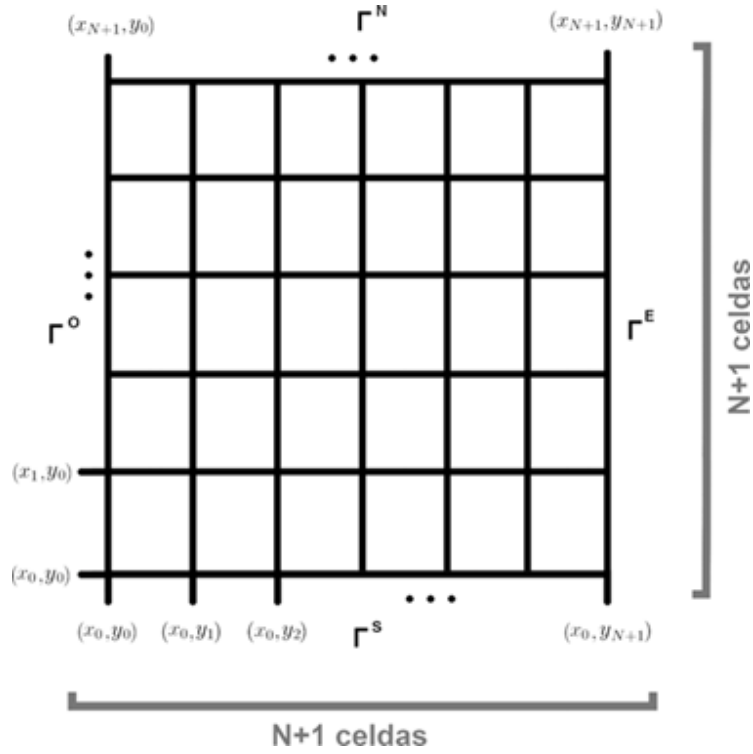


Figura 14: Mallado bidimensional rectangular para el problema de Poisson.

Para deducir el método de diferencias finitas, se escribe el problema (2.7) en los nodos de la malla

$$\begin{aligned}
 -\frac{\partial^2 u}{\partial x^2}(x_i, y_j) - \frac{\partial^2 u}{\partial y^2}(x_i, y_j) &= f(x_i, y_j) \quad 1 \leq i, j \leq N \\
 u(x_i, y_j) &= 0 \quad \forall (i, j) \in \Gamma_N^O \cup \Gamma_N^N \cup \Gamma_N^E \\
 -\frac{\partial u}{\partial y}(x_i, y_j) &= 0 \quad \forall (i, j) \in \Gamma_N^S.
 \end{aligned}$$

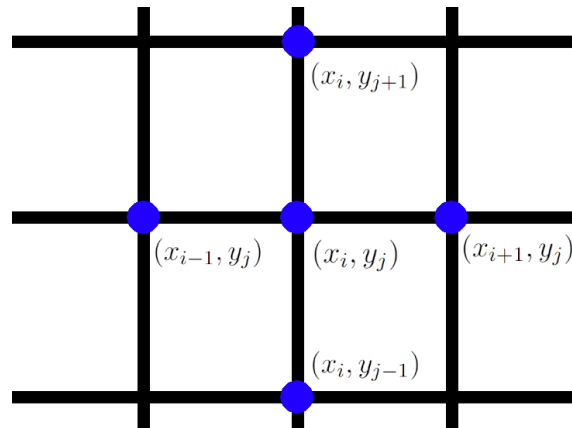


Figura 15: Detalle del esquema de 5 puntos o *5-points stencil* para los términos de orden dos.

De manera que, a partir de realizar dos veces el desarrollo de Taylor de orden 2 que ya se utilizó para la ecuación del calor, para los nodos interiores el método toma la forma

$$-\frac{\partial^2 u}{\partial x^2}(x_i, y_j) = \frac{-u(x_{i-1}, y_j) + 2u(x_i, y_j) - u(x_{i+1}, y_j)}{h_x^2} + \mathcal{O}(h_x^2),$$

$$-\frac{\partial^2 u}{\partial y^2}(x_i, y_j) = \frac{-u(x_i, y_{j-1}) + 2u(x_i, y_j) - u(x_i, y_{j+1}))}{h_y^2} + \mathcal{O}(h_y^2).$$

Para la frontera, los únicos nodos no determinados son los de la frontera sur, que vienen aproximados por

$$\frac{\partial u}{\partial y}(x_i, y_0) = \frac{u_{i,1} - u_{i,0}}{h_y} + \mathcal{O}(h_y) = 0.$$

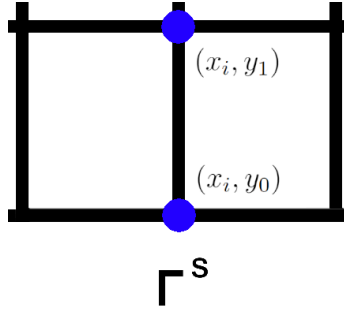


Figura 16: Detalle de los nodos en la frontera con condiciones Neumann.

Se pasa a definir el operador $\mathcal{L}_h : \mathbb{R}^{(N+2) \times (N+2)} \rightarrow \mathbb{R}^{N \times N}$ para todo $\mathbf{v} = (v_{i,j})_{0 \leq i,j \leq N+1} \in \mathbb{R}^{(N+2) \times (N+2)}$ por

$$(\mathcal{L}_N \mathbf{v})_{i,j} := \frac{-v_{i-1,j} + 2v_{i,j} - v_{i+1,j}}{h_x^2} + \frac{-v_{i,j-1} + 2v_{i,j} - v_{i,j+1}}{h_y^2}, \quad (i,j) \in \Omega_N,$$

$$(\mathcal{L}_N \mathbf{v})_{i,j} := \frac{v_{i,1} - v_{i,0}}{h_y}, \quad 1 \leq i \leq N,$$

$$(\mathcal{L}_N \mathbf{v})_{i,j} := v_{i,j}, \quad (i,j) \in \Gamma_N^O \cup \Gamma_N^N \cup \Gamma_N^E.$$

Este operador cumple

$$\mathcal{L}_N \mathbf{e} = \mathbf{r}, \quad e_{i,j} = 0 \quad \forall (i,j) \in \Gamma_N^O \cup \Gamma_N^N \cup \Gamma_N^E.$$

Se procede igual que antes analizando el orden del error de consistencia

$$r_{i,j} = \frac{2u(x_i, y_j) - u(x_{i-1}, y_j) - u(x_{i+1}, y_j)}{h_x^2} + \frac{2u(x_i, y_j) - u(x_i, y_{j-1}) - u(x_i, y_{j+1})}{h_y^2} \dots$$

$$\dots - f(x_i, y_j) = -\Delta u(x_i, y_j) + \mathcal{O}(h_x^2 + h_y^2) - f(x_i, y_j) = \mathcal{O}(h_x^2 + h_y^2), \quad (i, j) \in \Omega_N.$$

La única frontera en la que este error no es 0 es en la sur, en la que se tiene

$$r_{i,j} = \frac{u(x_i, y_1) - u(x_i, y_0)}{h_y} = \frac{\partial u}{\partial y}(x_i, y_0) + \mathcal{O}(h_y) = \mathcal{O}(h_y), \quad 1 \leq i \leq N.$$

De esta forma, con este planteamiento perdemos un orden en h_y debido a la frontera Neumann

$$\max_{0 \leq i, j \leq N+1} |r_{i,j}| = \mathcal{O}(h_x^2 + h_y).$$

Implementación

Se pasan a comentar los desafíos que presenta el problema de Poisson a nivel de código. Para ello, se utiliza el producto tensorial de Kronecker definido por

$$A \otimes B := \begin{pmatrix} a_{1,1}B & \cdots & a_{1,m}B \\ \vdots & \ddots & \vdots \\ a_{n,1}B & \cdots & a_{n,m}B \end{pmatrix} \in \mathbb{R}^{np \times mq}$$

donde A es una matriz $n \times m$ y B una matriz $p \times q$.

Además, se define la operación $\text{vec}(A) = (v_i) \in \mathbb{R}^{nm}$ con

$$v_{i+(j-1)N} := a_{i,j} \quad 1 \leq i \leq n, \quad 1 \leq j \leq m.$$

Estas dos operaciones cumplen las siguientes relaciones

$$\text{vec}(BA) = (A^\top \otimes I_N) \text{vec}(B) \quad \text{y} \quad \text{vec}(AB) = (I_N \otimes A) \text{vec}(B).$$

con I_N la matriz identidad $N \times N$.

A continuación, se reordenan las incógnitas del problema para obtener un sistema lineal cuadrado de $(N + 2)^2$ incógnitas. Se puede escribir directamente el esquema como

$$\frac{1}{h_x^2} D_{N+2} \mathbf{u} + \frac{1}{h_y^2} \mathbf{u} \tilde{D}_{N+2} = \mathbf{f},$$

donde $\mathbf{f} = (f(x_i, x_j))_{0 \leq i, j \leq N+1}$, $\mathbf{u} = (u_{i,j})_{0 \leq i, j \leq N+1}$ y $D_{N+2} := \text{tridiag}_{N+2}(-1, 2, -1)$. $\tilde{D}_{N+2}$ es esta misma matriz tridiagonal pero en la que se ha sustituido la primera fila por

$$\tilde{D}_{N+2}[0, :] = \frac{1}{h_y} [-1, 1, 0, \dots, 0],$$

para incluir las condiciones de contorno Neumann.

Aplicando el operador vec a la ecuación y usando la propiedad anterior deducimos que el MDF equivale a resolver el sistema lineal

$$A_{N+2} \text{vec}(\mathbf{u}) = \text{vec}(\mathbf{f})$$

donde la matriz $A_{N+2} := \frac{1}{h^2} (I_{N+2} \otimes D_{N+2} + D_{N+2} \otimes I_{N+2}) \in \mathbb{R}^{(N+2)^2 \times (N+2)^2}$ tiene la estructura

$$A_{N+2} = \frac{1}{h^2} \begin{pmatrix} 2I_{N+2} + D_{N+2} & -I_{N+2} & \cdots & 0 \\ -I_{N+2} & 2I_{N+2} + D_{N+2} & -I_{N+2} & \vdots \\ \vdots & -I_{N+2} & 2I_{N+2} + D_{N+2} & -I_{N+2} \\ 0 & \cdots & -I_{N+2} & 2I_{N+2} + D_{N+2} \end{pmatrix}$$

En las fronteras oeste, este y norte se imponen las condiciones de contorno Dirichlet mediante la técnica de penalización, que consiste en añadir un gran coeficiente $\lambda = 10^{16}$ en las filas de A_{N+2} correspondientes. Se procede anulando la fila k $A_{N+2}[k, :] \leftarrow 0$ completa, se fija $A_{N+2}[k, k] = \lambda$ y se ajusta $f_k = \lambda u^E(x_k, y_k)$.

Se reordena cada nodo (i, j) de la malla y se identifica globalmente por

$$k = i(N + 2) + j,$$

donde $i = 0, \dots, N + 1$ es el índice en x y $j = 0, \dots, N + 1$ el índice en y y que, en este caso concreto, este proceso se debe hacer para todo k en la frontera Dirichlet, pues cada fila k de A_{N+2} corresponde con la ecuación del nodo k en cuestión.

Finalmente, se resuelve la ecuación con un solver cualquiera, en **Python** o en **MATLAB**, para sistemas lineales y se desvectoriza la solución.

Estabilidad

Se mostrará a continuación un desarrollo breve y simplificado para poder entender en que consiste el análisis de estabilidad en este caso. Al no existir evolución temporal, en su lugar resolvemos el sistema lineal

$$A\mathbf{u} = \mathbf{f},$$

y, por tanto, el avance de los errores consiste en aplicar la inversa A^{-1} al vector del error de consistencia $\mathbf{r}$.

Gracias al **Principio del Máximo**, se sabe que A posee las propiedades de matrices necesarias para poseer inversa y que esta esté acotada. Más formalmente, el teorema dice que, sea A la matriz definida en los términos anteriores y un vector no nulo $\mathbf{v} \in \mathbb{R}^N$,

$$\mathbf{v}^\top A \mathbf{v} = \sum_{\text{celdas } (i,j)} \left(\frac{v_{i+1,j} - v_{i,j}}{h_x} \right)^2 + \left(\frac{v_{i,j+1} - v_{i,j}}{h_y} \right)^2 > 0. \quad (3.4)$$

A la matriz que cumple la ecuación (3.4) se le conoce como matriz definida positiva, presentado de manera más amplia en [6, 3.3.1 *The Discrete Maximum Principle*]. Seguidamente, con el **Lema 3.3** y el **Teorema 3.4** se consigue acotar en norma la matriz del método,

$$\|A^{-1}\|_\infty \leq C \Rightarrow \|\mathbf{e}\|_\infty = \|A^{-1}\mathbf{r}\|_\infty \leq \|A^{-1}\|_\infty \|\mathbf{r}\|_\infty \leq C \|\mathbf{r}\|_\infty.$$

que es el análogo a la desigualdad de estabilidad

$$\|e^n\| \leq C(\|e^0\| + \max_{1 \leq k \leq n} \|r^k\|).$$

Nota: En el libro se prueba la cota en la norma del supremo $\|\cdot\|_\infty$, pero usando la equivalencia de normas

$$\|vec(\mathbf{e})\|_2^2 = \frac{1}{(N+2)^2} \sum_{i=0, j=0}^{N+2, N+2} (e_{i,j})^2 \leq \|vec(\mathbf{e})\|_\infty^2 = \mathcal{O}((h_x^2 + h_y)^2).$$

3.2.3 Navier-Stokes

Para tratar mediante diferencias finitas el problema de Navier-Stokes presentado anteriormente, se utilizará el artículo [9], el cual presenta todo lo necesario desde el punto de vista del análisis matemático para poder comprender cómo se suelen tratar estos problemas.

No se pretende tratar el desarrollo entero con detalle, ya que lo presentado anteriormente se considera suficiente para entender en qué consiste el MDF. Sin embargo, es importante explicar algunos conceptos novedosos que, además de ser muy comunes y estar presentes en la bibliografía consultada, vale la pena conocer, así como comentar la forma matricial del problema, la cual se ha utilizado para traducir a **MATLAB** las explicaciones del artículo.

Antes que nada, cabe repetir que el artículo no trata exactamente este problema, sino que trabaja con la forma general de las ecuaciones de Navier-Stokes expuestas en (2.7), definiendo una velocidad y una presión iniciales que terminan dando como solución el vórtice de Taylor-Green.

La principal novedad consiste en introducir un término de *viscosidad artificial* para que el método sea más estable sin tener ningún cambio a nivel cualitativo, añadiéndolo a la ecuación de conservación de masa como $\frac{1}{\beta} \frac{\partial p}{\partial t}$. De este modo, el sistema que se resuelve en cada paso temporal se escribe como

$$\begin{cases} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} + \frac{\partial p}{\partial x} - \frac{1}{\text{Re}} \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = 0, \\ \frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} + \frac{\partial p}{\partial y} - \frac{1}{\text{Re}} \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right) = 0, \\ \frac{1}{\beta} \frac{\partial p}{\partial t} + \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0. \end{cases}$$

donde $\mathbf{u} = (u, v)$ define el campo de velocidad bidimensional, p la presión, $\frac{1}{\nu} = \text{Re}$ es lo que se conoce como **número de Reynolds** con ν la viscosidad cinemática y β un parámetro que controla la intensidad de la compresibilidad artificial. En la práctica, β se elige lo suficientemente grande para aproximar bien la incompresibilidad, pero sin llegar a provocar inestabilidades numéricas.

Se fija pues en este caso $\beta = 10^5$, pues a valores mayores no se observó una diferencia a nivel de resultados, de forma que este valor es suficiente. El desarrollo planteado en el artículo se recoge en el sistema matricial $3 \times (N_1 - 1) \times (N_2 - 1)$ que tiene la forma

$$\begin{bmatrix} A_1 & 0 & A_2 \\ 0 & A_1 & A_3 \\ \beta A_2 & \beta A_3 & I \end{bmatrix} \begin{pmatrix} u_1 \\ v_1 \\ p_1 \end{pmatrix} = b$$

donde

$$u_1 = [u_{1,1}^{n+1}, u_{2,1}^{n+1}, \dots, u_{N_1-1,N_2-1}^{n+1}]^T,$$

$$v_1 = [v_{1,1}^{n+1}, v_{2,1}^{n+1}, \dots, v_{N_1-1,N_2-1}^{n+1}]^T,$$

$$p_1 = [p_{1,1}^{n+1}, p_{2,1}^{n+1}, \dots, p_{N_1-1,N_2-1}^{n+1}]^T$$

y

$$A_1, A_2, A_3 \in \mathbb{R}^{(N_1-1)(N_2-1) \times (N_1-1)(N_2-1)}.$$

Estas matrices recogen la siguiente información:

- A_1 : recoge la discretización en las ecuaciones de momento, que incluye el término temporal u_t y término viscoso o difusivo $-\frac{1}{\text{Re}}(\partial_x^2 + \partial_y^2)$ para u y v .
- A_2 : corresponde al operador de gradiente de presión en la dirección x .
- A_3 : análoga a A_2 , pero para el operador de gradiente de presión en la dirección y y la componente v .

Este esquema de diferencias finitas es semi-implícito, como el de Crank-Nicolson para la ecuación del calor, lo cual mejora la estabilidad del esquema. Para su implementación en **MATLAB** se han utilizado técnicas como el producto de Kronecker o las matrices vacías que ya se comentaron durante los métodos anteriores, siendo como se ha repetido en varias ocasiones la construcción de la matriz el mayor desafío técnico de estos métodos.

Capítulo 4

Conceptos Básicos de Machine Learning

El **machine learning** o aprendizaje automático conforma un campo de estudio que se remonta a la década de 1950, cuando se exploraron por primera vez métodos computacionales capaces de emular procesos de aprendizaje humano. Fue durante esa época en la que se definió el concepto básico de perceptrón y se sentaron las bases teóricas de las redes neuronales, aprovechando el auge de los ordenadores digitales para implementar modelos lineales de clasificación.

En la última década, el aprendizaje automático ha vivido una auténtica revolución gracias al desarrollo de arquitecturas especializadas, como los famosos *LLM* o modelos de lenguaje grandes, entrenados con enormes volúmenes de texto para comprender, predecir y generar lenguaje de manera coherente (ChatGPT), y las *redes convolucionales*, especialmente diseñadas para procesar datos con estructura de cuadrícula y siendo cruciales para el procesamiento y generación de imágenes (Midjourney).

La teoría de este apartado está fuertemente basada en [7], el cual es un recurso bibliográfico muy recomendable en caso de querer iniciarse o profundizar en el tema.

4.1 Fundamentos

Lo primero que se hace necesario es definir qué es un **modelo** y en qué consiste modelizar un problema. Se define un modelo computacional como una representación matemática que establece relaciones entre variables de entrada y salida, permitiendo predecir comportamientos a partir de datos. En el contexto del machine learning, se concibe que un modelo aprende patrones estadísticos de un conjunto de entrenamiento para generalizar sobre ejemplos no vistos.

Este trabajo se centra particularmente en el aprendizaje automático **supervisado**. Esto es, sea el conjunto de **datos de entrenamiento** la colección $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, en la que $\mathbf{x}^{(i)} \in \mathbb{R}^m$ representa las m características de la **muestra** i e $y^{(i)}$ el valor objetivo, se define la **función de pérdida** $\mathcal{L}(\theta) = \ell(f_{\theta}(\mathbf{x}), y)$ como la distancia en cierta *métrica* ℓ entre el valor predicho por el modelo f (con parámetros θ) y el valor objetivo.

El proceso de aprendizaje consiste pues en ajustar los parámetros internos del modelo minimizando esta función de pérdida, que cuantifica la discrepancia entre las predicciones y las etiquetas verdaderas. A cada iteración de este proceso se le llamará **paso** y a un conjunto de cierto número de pasos se le llamará por conveniencia **época**.

Bajo este paradigma, una **red neuronal artificial** (RNA) se concibe como un ensamblaje de unidades de procesamiento, llamadas neuronas artificiales, organizadas en **capas** y conectadas mediante **pesos**. Cada neurona recibe un *vector de entradas*, realiza una combinación lineal de las mismas y aplica una función de activación no lineal para producir una *señal de salida*.

En este trabajo, este ensamblaje se dispone en una arquitectura en la que las capas se suceden sin realimentación, constituyendo lo que se conoce como *red feedforward*, las cuales únicamente y exclusivamente propagan la información en un único sentido.

Para hablar con propiedad, se deberá distinguir entre **parámetros** del modelo, que corresponden a los pesos y sesgos que se ajustan automáticamente durante el proceso de optimización, e **hiperparámetros**, que deben fijarse antes del entrenamiento y controlan la capacidad de representación y la dinámica de convergencia del algoritmo, como lo pueden ser el número de capas y de neuronas de la red.

El **perceptrón** [8] constituye el modelo más elemental de una neurona artificial. Se define como una función lineal seguida de una función de activación escalón, de modo que recibe un vector de entrada $\mathbf{x} = [x_1, x_2, \dots, x_n]$ y produce una salida binaria $y \in \{-1, 1\}$. Matemáticamente, se introduce un vector de pesos $\mathbf{w} = [w_1, w_2, \dots, w_n]$ y un sesgo b , formándose la combinación lineal

$$z = \mathbf{w}^{\top} \mathbf{x} + b,$$

y aplicando a continuación la función escalón

$$y = \begin{cases} 1, & \text{si } z \geq 0, \\ -1, & \text{si } z < 0. \end{cases}$$

Con esta definición, el perceptrón único separa el espacio de entrada mediante un hiperplano $\mathbf{w}^\top \mathbf{x} + b = 0$, clasificando los puntos de un lado como positivos y del otro como negativos. Por tanto, si los datos son separables por un hiperplano, el perceptrón simple será capaz de distinguir ambos conjuntos.

El proceso de entrenamiento de este modelo sencillo se basa en la regla de aprendizaje del perceptrón, que ajusta iterativamente los pesos y el sesgo para corregir las clasificaciones erróneas en un conjunto de datos linealmente separable. En cada iteración y para cada muestra $(\mathbf{x}^{(i)}, y^{(i)})$, se actualiza

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y^{(i)} - \hat{y}^{(i)}) \mathbf{x}^{(i)}, \quad b \leftarrow b + \eta (y^{(i)} - \hat{y}^{(i)}),$$

donde $\hat{y}^{(i)}$ es la predicción actual, $y^{(i)}$ la etiqueta verdadera y η la **tasa de aprendizaje**.

4.2 El perceptrón multicapa

El perceptrón simple, al basar su decisión en una combinación lineal de las entradas seguida de una función escalón, solo es capaz de resolver problemas linealmente separables, lo que revela su incapacidad para capturar patrones no lineales.

Esta restricción origina un sesgo elevado al aproximar funciones más complejas y lo hace especialmente vulnerable al ruido y a estructuras de datos intrincadas. Tales limitaciones motivaron la incorporación de capas ocultas y de funciones de activación diferenciables, de modo que la red pueda aprender representaciones intermedias y modelar relaciones no lineales, sentando así las bases del perceptrón multicapa.

El **perceptrón multicapa (MLP)** se concibe como un conjunto de unidades de procesamiento que transforman información mediante operaciones lineales y no lineales encadenadas. En sus primeros desarrollos, cada neurona realizaba una suma ponderada de sus entradas seguida de una función de activación, permitiendo a las RNAs captar patrones no lineales.

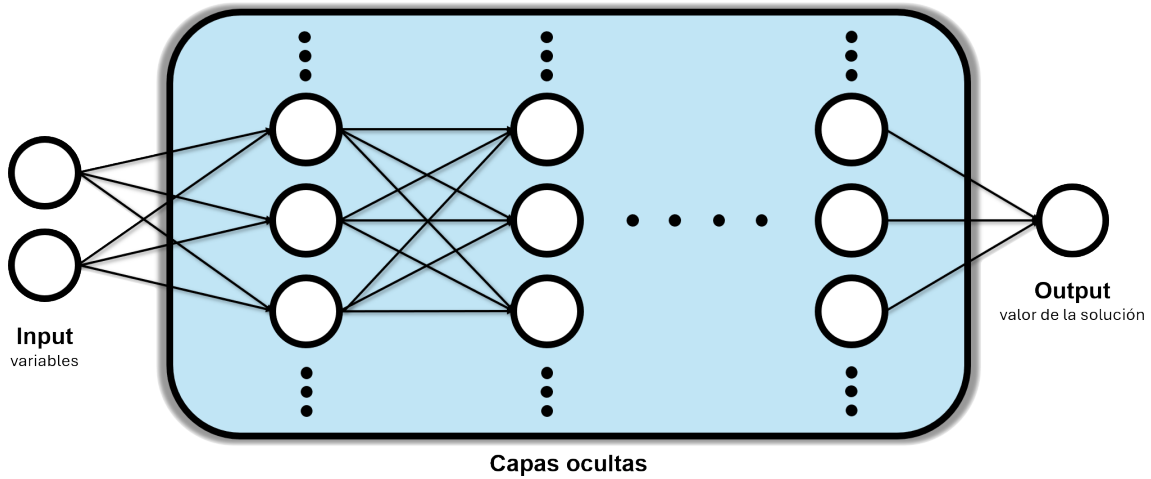


Figura 17: Diagrama genérico de los MLPs con capas completamente conectadas.

De este modo, se introdujo la noción de capas ocultas, ubicadas entre la capa de entrada y la de salida, permitiendo a la red aproximar funciones arbitrarias siempre que contara con suficientes unidades y profundidad.

Teorema (Aproximación Universal): Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}$ una función continua definida sobre un subconjunto compacto $K \subset \mathbb{R}^n$. Entonces, para todo $\varepsilon > 0$ existe una red neuronal de una sola capa oculta (es decir, un perceptrón multicapa con activación no lineal arbitraria) con un número finito de neuronas en la capa oculta tal que, al denotar su salida por $\hat{f}(x)$, se cumple

$$\sup_{x \in K} |\hat{f}(x) - f(x)| < \varepsilon.$$

El verdadero problema a la hora de llevar este método a la práctica es que, ni este teorema ni ningún otro, da una idea acerca de cuántas de estas neuronas se necesitan para resolver un problema concreto.

Es por esto que, dentro de las redes feedforward, el MLP se identifica como la subclase elemental. Capa de entrada recibe los valores de las características originales, las capas ocultas generan representaciones intermedias de mayor abstracción y la capa de salida emite la predicción final. Cada capa lineal se expresa mediante $\mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$, con $\mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)})$ la **función de activación** y ℓ el índice de la capa, donde ϕ puede ser, por ejemplo, la famosa función *Rectified Linear Unit* (**ReLU**), la *tangente hiperbólica* ($\tanh$),

$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ contiene en cada fila los pesos que conectan todas las neuronas de la capa $\ell - 1$ con una neurona de la capa ℓ , mientras que $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$ es un vector cuyos componentes son los sesgos de cada neurona de la capa ℓ .

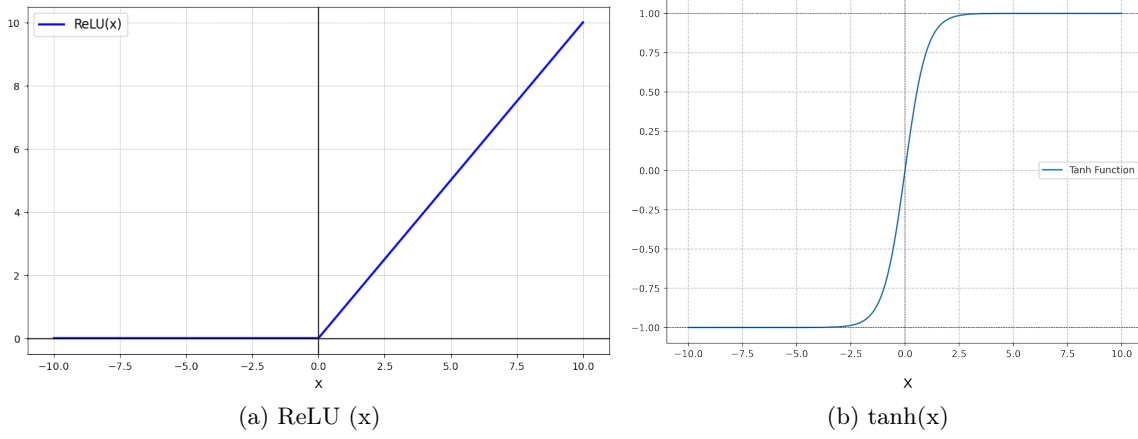


Figura 18: Ejemplos de funciones de activación importantes para este proyecto.

Por todo esto, la RNA que se presentará y se utilizará a lo largo del trabajo será un MLP, de forma que no se hará distinción entre ambos términos. Para minimizar la función de pérdida mencionada anteriormente, se hará uso del algoritmo de **retropropagación** del error, el cual consiste en actualizar todos los parámetros del modelo haciendo uso de un **optimizador** basado en descenso de gradiente.

De esta forma, el entrenamiento de la red neuronal consta de tres fases fundamentales

1. Propagación hacia adelante: Se introducen las entradas $\mathbf{x}$ en la capa de entrada y se calcula sucesivamente la salida de cada capa

$$\mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)}),$$

hasta obtener la predicción final $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$. A continuación se evalúa la función de pérdida $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$.

2. Retropropagación: Se calcula el gradiente $\nabla_{\theta} \mathcal{L}$. Aplicando la regla de la cadena se obtiene, para cada peso y sesgo,

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} \frac{\partial \mathbf{a}^{(\ell)}}{\partial \mathbf{z}^{(\ell)}} \frac{\partial \mathbf{z}^{(\ell)}}{\partial W^{(\ell)}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} \frac{\partial \mathbf{a}^{(\ell)}}{\partial \mathbf{z}^{(\ell)}}.$$

3. Actualización de parámetros: Una vez disponibles los gradientes, el optimizador aplica la regla de descenso de gradiente

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t),$$

donde $\boldsymbol{\theta}$ agrupa todos los pesos $W^{(\ell)}$ y sesgos $\mathbf{b}^{(\ell)}$ y η es la **tasa de aprendizaje**.

Se ha observado que una tasa de aprendizaje demasiado alta induce oscilaciones e inestabilidad, mientras que una demasiado baja ralentiza el entrenamiento. Para ajustar este parámetro de forma dinámica, se incorporan **planificadores** o schedulers que pueden reducir gradualmente la tasa de aprendizaje de diversas maneras.

4.3 Implementación en el campo de las EDPs

Se definirán los dos conceptos fundamentales que marcarán la manera en la que se procederá para analizar numéricamente las ecuaciones mediante RNAs, sobre todo para enmarcarlos dentro de la literatura y saber que nombre reciben. La web de **MATLAB** [16] ofrece un recurso visual y una explicación muy parecida a esta, por si se requiere más información.

Se conoce como **Physics-Informed Neural Network** (PINN) a toda red neuronal cuya función de pérdida incorpore términos con sentido físico, siendo en este caso

$$\mathcal{L}(\theta) = \ell(\mathcal{N}(f_{\theta}(\mathbf{x}_{\text{EDP}})), y_{\text{EDP}}) + \ell(f_{\theta}(\mathbf{x}_{\text{IC}}), y_{\text{IC}}) + \ell(f_{\theta}(\mathbf{x}_{\text{BC}}), y_{\text{BC}}).$$

Se seguirá hablando de ellas como RNAs o MLPs para no complicar más la terminología, pero es importante conocer el nombre que reciben. Aquí los subíndices EDP se utilizan para etiquetar a los puntos de la malla de entrenamiento que se utilizan para verificar la ecuación.

Es decir, para hallar esa diferencia primero se evalúa la red f_{θ} en esos puntos, después se le aplica el operador diferencial de la EDP $\mathcal{N}$ (por ejemplo $\partial_t - \alpha \partial_{xx}^2$ para el calor) salido de pasar todos los términos de la ecuación a un lado y por último hallar su distancia con y_{EDP} a través de la métrica, que típicamente es un vector de ceros.

En la mayoría de ocasiones contaremos tanto con un término $\mathcal{L}_{\text{BC}}$ que controla el

error de las condiciones de contorno a partir de un conjunto de puntos generados en la frontera y análogamente para $\mathcal{L}_{IC}$ pero con las condiciones iniciales. Así pues, en general las funciones de pérdida se verán así

$$\mathcal{L}_{EDP} + \lambda_{BC}\mathcal{L}_{BC} + \lambda_{IC}\mathcal{L}_{IC}, \quad (4.1)$$

donde λ_{BC} y λ_{IC} serán hiperparámetros que permitirán controlar la influencia de cada componente del error en la función de pérdida. Lo último a comentar es que la métrica a utilizar será el MSE ya presentado antes, para que sea consecuente con el análisis de convergencia realizado para los métodos de diferencias finitas.

Para definir el conjunto que se utilizará durante el entrenamiento se utilizará el **Deep Galerkin Method** (DGM) [17], que es el procedimiento que hace realmente poderosas a este tipo de redes. Este no es más que una variante de las PINNs que busca minimizar $\mathcal{L}$ mediante el **muestreo aleatorio** de los puntos en el dominio correspondiente para cada paso.

Es decir, en cada iteración se generan los conjuntos $\mathbf{x}_{EDP}$, $\mathbf{x}_{IC}$ y $\mathbf{x}_{BC}$ aleatoriamente y el modelo se auto ajusta con respecto a los errores de esta malla. Esto garantiza que, si se realizan los suficientes pasos y se entrena la RNA los máximos puntos del dominio posibles, se obtendrá una muy buena solución para todos y cada uno de los puntos de este, de forma que se podría tratar a nuestra red neuronal como una función continua al uso que se aproxima a la solución en ese dominio.

4.4 Comentarios y variantes del modelo

En este apartado detallan algunos conceptos clave del proceso de entrenamiento y ajuste de la red neuronal aplicado a la resolución de EDPs, comenzando por la elección del optimizador, pasando por la presentación del concepto de regularización y sobreajuste y explorando variaciones tanto en las funciones de activación como en los pesos asociados a los términos de condiciones iniciales y de contorno, las cuales se ilustran a continuación.

Para la optimización de los parámetros de la red se ha utilizado el optimizador **Adam**, que permite ajustar tasas de aprendizaje individuales para cada parámetro y garantiza una convergencia más rápida y estable que la ofrecida por el descenso de gradiente clásico,

mediante el uso de momentos móviles (primer momento = media de gradientes, segundo momento = media de cuadrados de gradientes).

Para ir modificando esta tasa de aprendizaje a medida que el modelo avanza, se ha utilizado un planificador basado en **milestones** o metas, de manera a medida que se alcanzan ciertos números de pasos esta se reduce en un factor γ fijo.

Cabe aclarar que ni el optimizador ni el planificador se incrustan dentro de las técnicas de **regularización** (para controlar que ciertas neuronas tengan una influencia desmedida y el modelo memorice) ni ayudan excesivamente a la localización del mínimo global de la función de pérdida.

Estos se limitan a implementar un conjunto de reglas de actualización de los parámetros basadas en información local de los gradientes, con el objetivo de descender por pendientes favorables en el paisaje de pérdida convergiendo con alta probabilidad a mínimos suaves debido a la alta dimensionalidad de los espacios de parámetros, lo cual es muy importante, pero no garantiza la convergencia global.

Así pues, si bien es verdad que el uso de los momentos presentes en Adam, que introduce inercia en la dirección de descenso ayudando a superar pequeños valles y pendientes cambiantes, del planificador y la aleatoriedad de los datos de entrada ayudan en este sentido, no evitan con total certeza problemas como el ya mencionado o el **overfitting** (sobreajuste del modelo al conjunto de datos de entrenamiento) o el estancamiento en un mínimo local.

Algunas técnicas que sí son específicas para esta tarea son el *dropout*, el cual apaga ciertas neuronas de manera aleatoria para que el modelo distribuya el aprendizaje a través de toda la red, o el *weight decay*, que penaliza los pesos excesivamente grandes multiplicándolos por un factor. Ninguno de estos métodos se ha explorado en este trabajo no ha sido necesario en vista de los resultados.

El estudio de la regularización requiere mucho más que los comentarios realizados, pero como las ecuaciones tratadas no han dado lugar a ninguna de las problemáticas que estos resuelven, no se ha considerado necesario profundizar mucho más para facilitar la lectura.

Se han explorado dos tipos de variaciones principales, las funciones de activación y los pesos λ_{IC} y λ_{BC} presentados en la ecuación (4.1), para mostrar cómo estos elementos pueden influir fácilmente en los resultados obtenidos.

Se utilizaron las funciones nativas `nn.ReLU` y `nn.Tanh` de `PyTorch`, una librería de `Python` en la cual se basan también las redes neuronales definidas. Además, se probó por cuenta propia la función seno

```
class Sine(nn.Module):
    def __init__(self):
        super().__init__()
    def forward(self, x):
        return torch.sin(x)
```

surgida del interés por probar una senoide pura [10] y de dar un ejemplo sobre cómo se puede definir una función de activación cualquiera en `Python`.

La modificación de los pesos de cada término se hace necesaria debido al alto número de casuísticas que se pueden encontrar en el mundo de las EDPs, siendo algunas especialmente demandantes con respecto a las condiciones iniciales y de contorno, como pueden ser los fenómenos caóticos como el sistema de Lorenz mencionado al inicio del trabajo, en los cuales por definición pequeñas variaciones inducen cambios radicales en la forma de la solución.

Comentar que, a la hora de calcular los errores durante el entrenamiento, hay que tener extremo cuidado con que los vectores tengan las mismas dimensiones, ya que `Python` diferencia entre la dimensión `[N, 1]` y `[N]`, no importando especialmente cual se use siempre y cuando se sea consistente.

Por último, se explorarán las facilidades de las que gozan las RNAs en términos de incluir “parámetros nuevos”, como lo será el coeficiente de difusión térmica α en la ecuación del calor, y de obtener la solución en mallas no rectangulares, aproximándose la solución de la ecuación del calor bidimensional en una malla triangular.

Ni que decir tiene que estas dos variaciones no son tan fácilmente implementables en el terreno de los MDFs desde el punto de vista teórico, pues por ejemplo definir una malla

para una forma arbitraria y a partir de ella moldear el método tiende a ser una tarea demasiado intrincada.

Si bien para la ecuación unidimensional del calor se conoce la solución exacta para cada α , la cual se utilizará para comparar nuestra red, para la ecuación bidimensional en malla triangular no se tiene una metodología para saber si la red está consiguiendo converger.

Para solucionar en este último caso este problema, se comparará la red con la solución dada por un *Add-On* o extensión de MATLAB muy sencilla de utilizar y comprender, la cual es **Partial Differential Equation Toolbox** [11] y consiste, como se puede ver en la figura 19, en definir una poligonal indicando los vértices, pasarle un parámetro que marca la precisión del mallado (pequeños triángulos, una posible solución para ciertas figuras del problema mencionado dos párrafos atrás) e indicarle los coeficientes necesarios para definir la ecuación del calor.

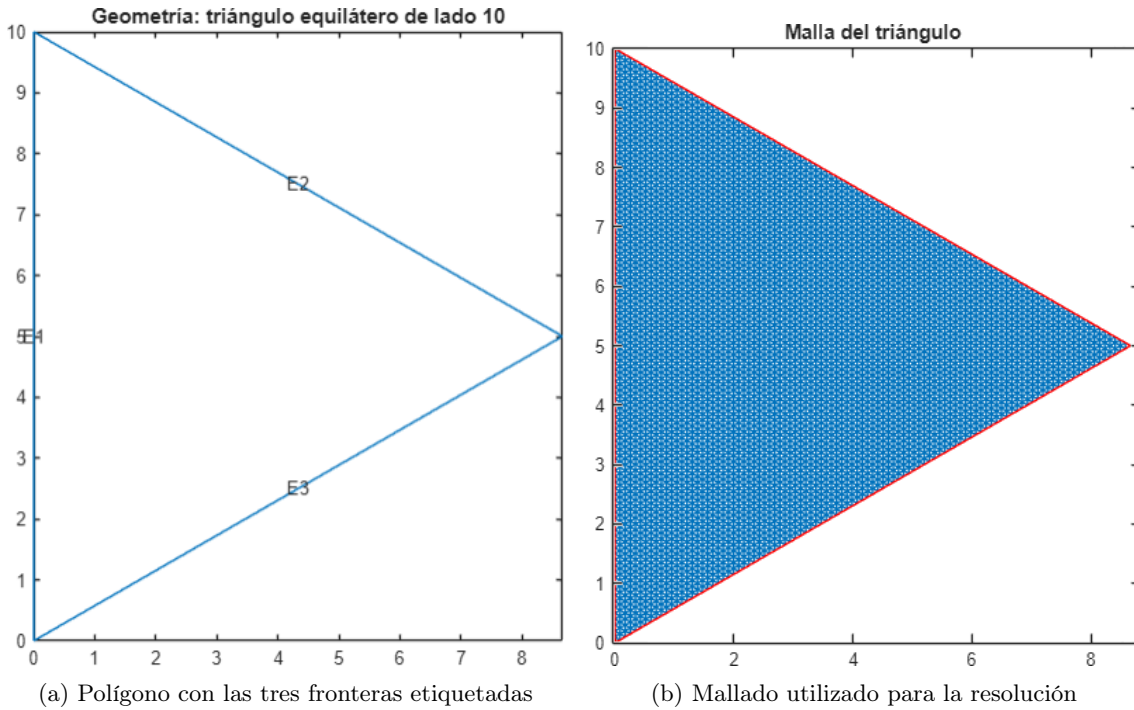


Figura 19: Triángulos equiláteros de lado L construidos con este procedimiento.

Estos coeficientes serán $m = 0$, $d = 1$, $c = 1$, $a = 0$ y $f = 0$, de acuerdo a la ecuación general que la extensión es capaz de resolver mediante el solver `solvepde`

$$m \frac{\partial^2 u}{\partial t^2} + d \frac{\partial u}{\partial t} - \nabla \cdot (c \nabla u) + au = f.$$

La ecuación bidimensional de calor se trató con condiciones de contorno Dirichlet nulas a lo largo de toda la frontera de este triángulo representada en la figura 19 y como condición inicial se eligió una exponencial contenida convenientemente en el interior de este, para que sea lo más visual posible en el futuro.

Capítulo 5

Resultados

5.1 Comparativa entre redes neuronales

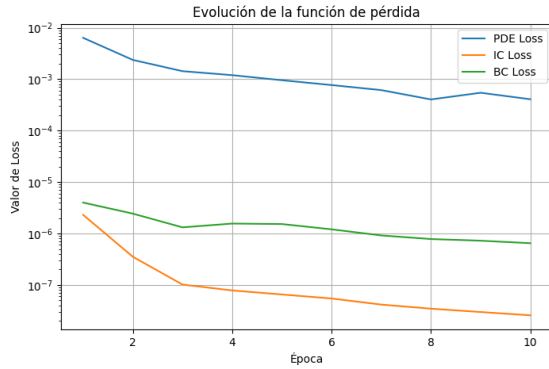
Como se comentó anteriormente, para cada ecuación se han definido variaciones del modelo de redes neuronales general. Se hace necesario presentar el rendimiento entre estas cuatro estructuras distintas para tener un argumento de peso con el que elegir la mejor de todas ellas y compararla con los resultados del método de diferencias finitas.

Concretando lo anteriormente expuesto, se definirán los siguientes modelos que representarán las variantes que se han estudiado:

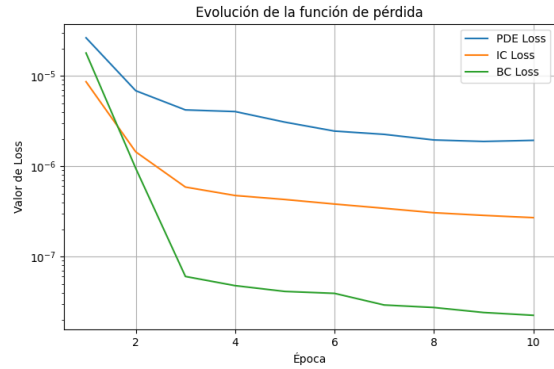
- (I) **Modelo 1:** utiliza función de activación seno y pesos no triviales en los distintos términos de la función de pérdida, representados por λ_{IC} y λ_{BC} distintos de 1, para potenciar la influencia de las condiciones iniciales y de contorno (carpeta `/lambdas`).
- (II) **Modelo 2:** emplea función de activación sinusoidal.
- (III) **Modelo 3:** incorpora función de activación tangente hiperbólica.
- (IV) **Modelo 4:** utiliza función de activación ReLU.

Se procede pues a presentar la evolución de los errores a lo largo de las épocas, todas ellas de 2500 pasos. Se recomienda mirar con algo de detenimiento el eje vertical de cada gráfica, pues no es el mismo en todas ellas.

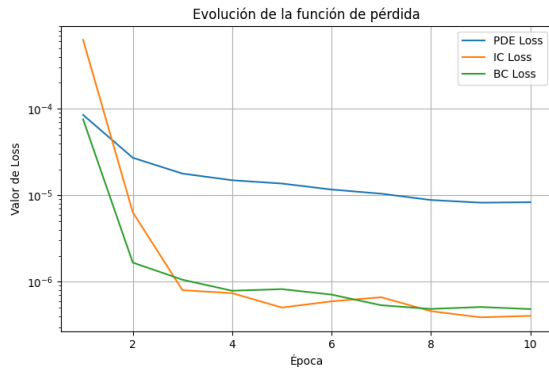
Ecuación del calor



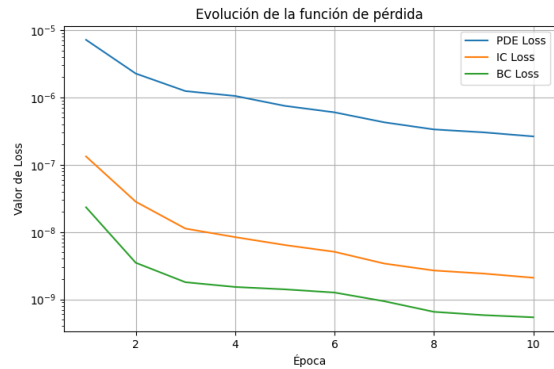
(a) · Modelo 1 - 115 segundos/época



(b) · Modelo 2 - 113 segundos/época



(c) · Modelo 3 - 107 segundos/época



(d) · Modelo 4 - 69 segundos/época

Figura 20: Gráficas que muestran la evolución del MSE para la ecuación del calor.

Tras haber alcanzado las 10 épocas, el cual es el número estándar que se utiliza para detener el proceso de entrenamiento y que todos los modelos realicen un número uniforme de pasos, el **Modelo 2** ha sido el que mejor ha rendido. De estas gráficas 20 cabe mencionar una serie de cosas que no se volverán a repetir ya que son recurrentes para todas las ecuaciones.

La primera es que, como cabía esperar, tanto los errores en la frontera (BC) como los errores de la condición inicial (IC) efectivamente se reducen más rápidamente en el **Modelo 1** debido a la imposición de pesos λ_{BC} y λ_{IC} mayores que 1. Esto solo es útil en problemas que lo necesiten fuertemente, pues como también se puede ver, estos pesos causan un decaimiento más lento del error asociado a la EDP, el cual no puede descuidarse pues es lo que debe verificar todo el dominio.

La segunda es que uno podría pensar que el **Modelo 4** es muy superior al resto viendo la gráfica. Sin embargo, si bien es verdad que el modelo cumple las condiciones de contorno e iniciales **en este caso**, las métricas del EDP Loss parecen no ser correctas ya que, a la hora de evaluar y obtener la gráfica 24, devuelve una solución que no varía en el tiempo, lo cual es incorrecto.

Esto permite comprobar la superioridad de las funciones de activación trigonométricas en este tipo de problemas, como se recoge de manera muy acertada en el ya mencionado [10].

Ecuación del calor con α libre

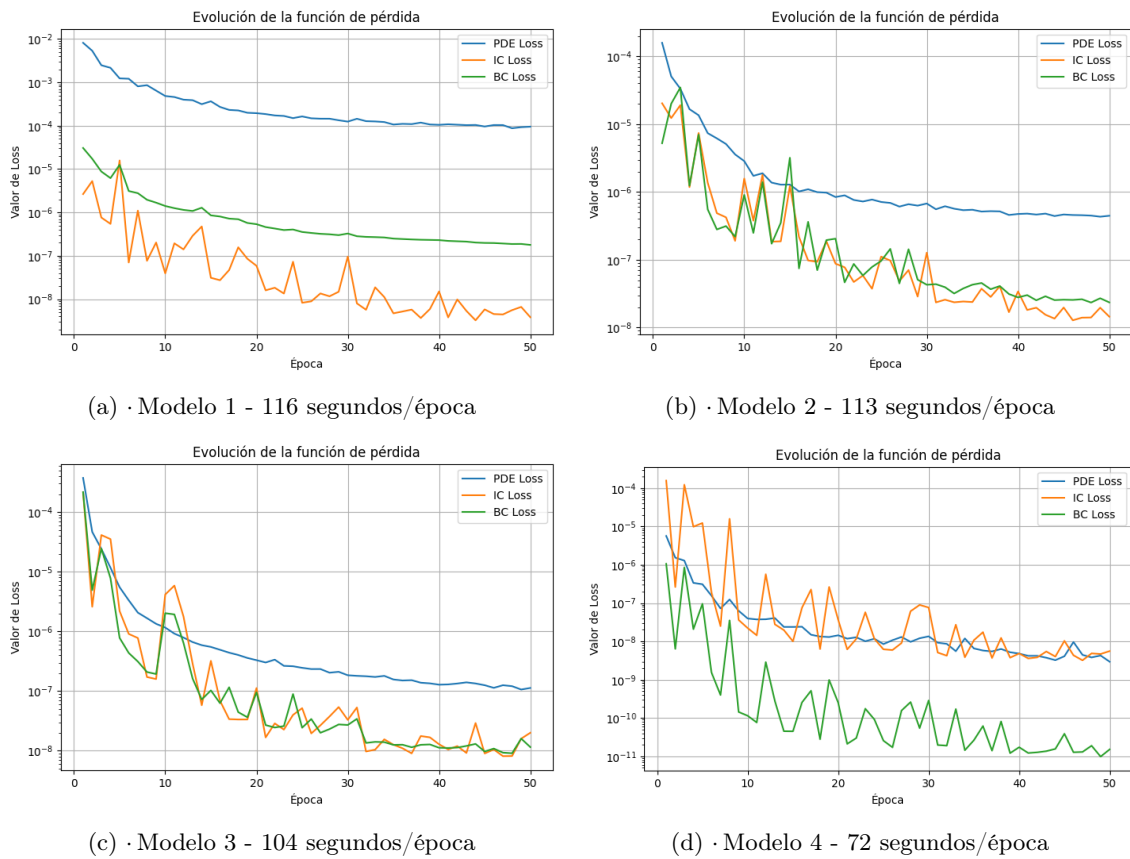
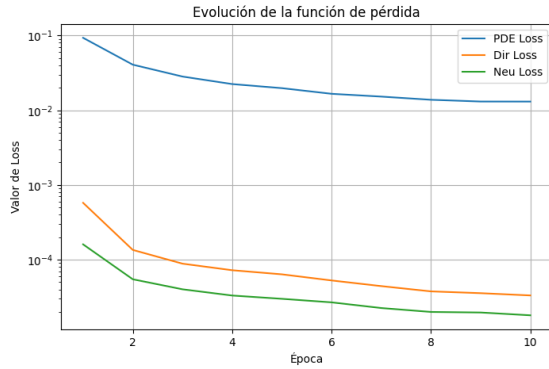


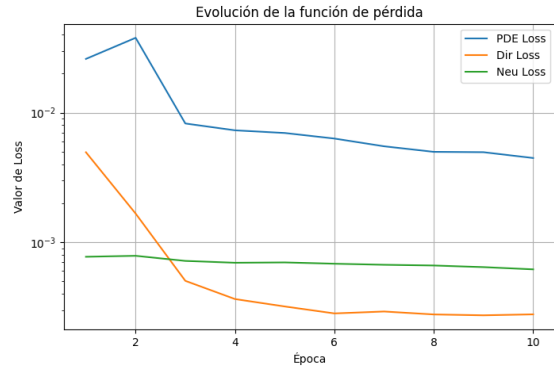
Figura 21: Gráficas que muestran la evolución del MSE para la ecuación del calor con α libre.

Tras haber alcanzado las 50 épocas, se puede apreciar [21] que el **Modelo 3** ha sido el que mejor ha rendido. Como se puede apreciar, el tiempo de entrenamiento por época no ha aumentado drásticamente con la inclusión de este nuevo parámetro.

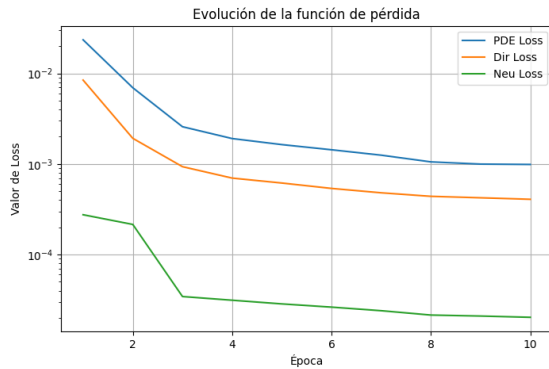
Ecuación de Poisson



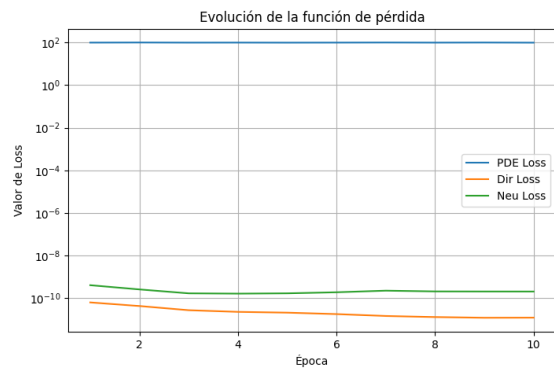
(a) · Modelo 1 - 126 segundos/época



(b) · Modelo 2 - 125 segundos/época



(c) · Modelo 3 - 116 segundos/época



(d) · Modelo 4 - 78 segundos/época

Figura 22: Gráficas que muestran la evolución del MSE para la ecuación de Poisson.

Tras haber alcanzado las 10 épocas, el **Modelo 1** ha sido el que mejor ha rendido pese a ser visualmente peor que 1 y 2 en la figura 22. El criterio de elección de cuál es el mejor de los cuatro es su evaluación en un conjunto enorme de puntos, 4 millones en este caso, dando lugar a un error de hasta un orden menor al del resto. En este caso, el error asociado a las condiciones iniciales desaparece, junto con su peso, debido a que la ecuación no evoluciona en el tiempo. En su lugar, se separa el error asociado a las condiciones de contorno en dos, uno para Dirichlet y otro para Neumann.

Ecuaciones de Navier-Stokes

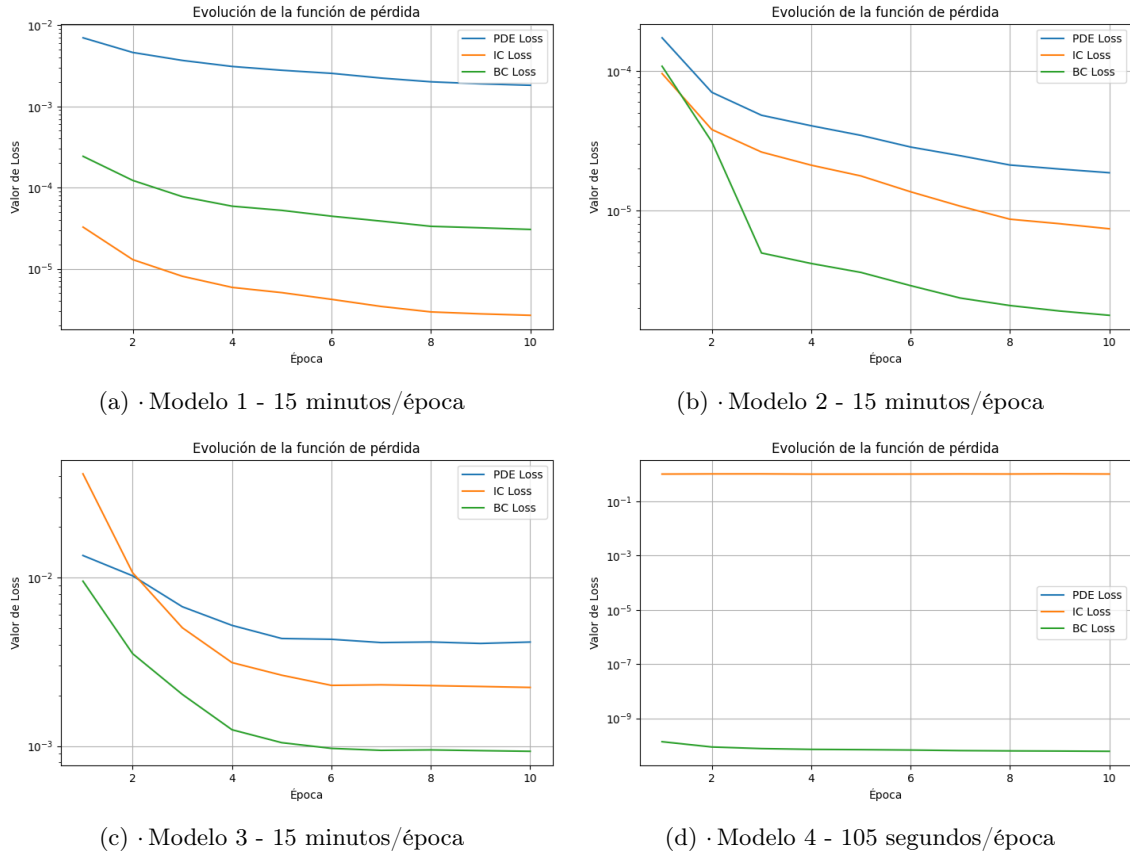


Figura 23: Gráficas que muestran la evolución del MSE para la ecuación de Navier-Stokes.

Tras haber alcanzado las 10 épocas, el **Modelo 1** vuelve a ser el que mejor ha rendido, evidenciando la importancia de mantener un error bajo en la frontera, no solo por el simple hecho de tener poco error en estos puntos, sino porque por mucho que la red neuronal verifique la ecuación (otros modelos presentan una mejor métrica en factor, como podemos apreciar en la figura 23) y sea una solución aproximada buena, se requiere que esta sea la *solución concreta* que se busca, una solución única garantizada por estas condiciones.

5.2 Diferencias entre métodos clásicos y modernos

En este apartado mostraremos las soluciones que se han obtenido a través de la mejor red neuronal seleccionada anteriormente y del método de diferencias finitas para cada ecuación. Además, se presentarán ciertos datos interesantes que permitirán definir una utilidad clara para ambos métodos. Obviamente, todo fue ejecutado en el mismo sistema, el cual cuenta con 32GB de memoria RAM (Memoria de Acceso Aleatorio), una CPU (Unidad de Procesamiento Central) AMD Ryzen 5 5600X y una GPU (Unidad de Procesamiento Gráfico) RTX 3060 Ti 8GB con acceso a CUDA (arquitectura diseñada para operaciones específicas en paralelo).

Omitiremos las imágenes de las soluciones exactas, pues son **visualmente** idénticas a las soluciones dadas tanto por las RNAs como por los MDFs. De igual forma, estas siempre son mostradas en los correspondientes **notebooks** (archivos con formato `.ipynb` en el recurso online, Github, que contienen todo el código utilizado) con sufijo `_NN`.

5.2.1 Resultados para la ecuación del calor

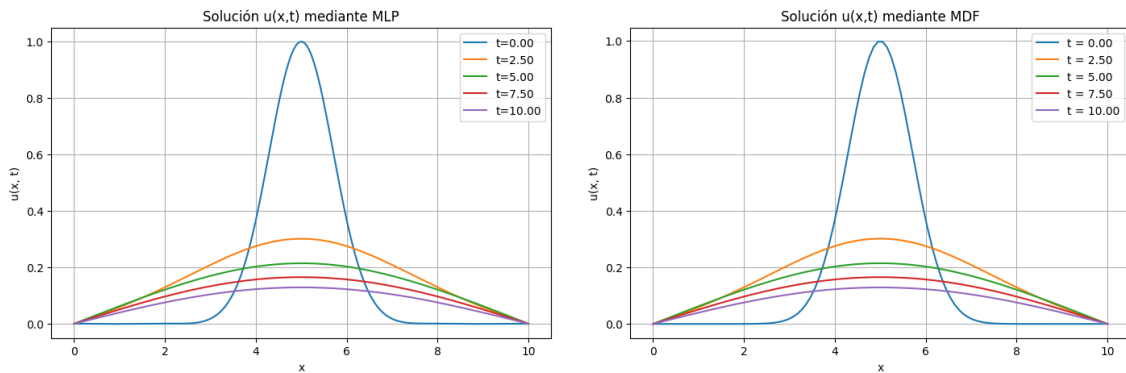


Figura 24: Soluciones aproximadas para la ecuación del calor.

Como se puede apreciar en la figura 24, ambos modelos resuelven satisfactoriamente este sencillo problema en una malla de 10,000 puntos tanto en tiempo como en espacio, es decir, un total de 100,000,000 de puntos a evaluar. De los datos presentados anteriormente se obtiene que el tiempo de entrenamiento de la red (8 capas ocultas de 100 neuronas cada una) fue de casi 19 minutos, frente a los 27 segundos del método de diferencias finitas.

Cabe destacar que esta red parece ser lo suficientemente amplia como para aproximar de manera satisfactoria la ecuación del calor, pudiendo explorarse con arquitecturas reducidas con menos capas y, por ende, menor tiempo de entrenamiento. Esto está soportado porque, al introducir α como parámetro extra, se observa que esta red no presenta mayor problema en aproximar la solución en el mismo número de épocas.

Para los tiempos mencionados, los métodos presentan unas desviaciones con respecto a la solución exacta de

Tiempos (segundos)	MSE (RNA)	MSE (MDF)
0	2.7×10^{-7}	1.2×10^{-24}
2.5	1.6×10^{-8}	5.4×10^{-19}
5	1.9×10^{-8}	2.7×10^{-19}
7.5	1.1×10^{-8}	3.5×10^{-19}
10	8.4×10^{-9}	3.8×10^{-19}

La diferencia es abrumadora, pero igualmente hay que tener en cuenta dos cosas. La primera es que la red neuronal es capaz de mantener de manera consistente estos errores obtenidos para la misma malla del MDF, que igualmente son muy buenos, a lo largo de todo el dominio. Esta es la primera cosa positiva en la que destacan las redes neuronales por encima del resultado que ofrece diferencias finitas, sacrificando precisión (a veces innecesaria, pues no suele hacer falta tener precisión a octavo decimal) a cambio de tener una solución definida en todo punto.

La segunda es que el método de diferencias finitas para esta ecuación tan simple está muy optimizado y funciona muy bien incluso en mallas gigantescas, pues recordemos que se está haciendo uso del esquema de Crank-Nicolson para ganar aún más orden. En general, no es computacionalmente posible ejecutar este método en la malla que se desee para cualquier ecuación.

Aquí se podría pensar que esto no es un problema, ya que no hacen falta tantísimos puntos para ningún tipo de uso, lo cual es cierto. Pero cuando se ha estado repitiendo tanto que la ecuación del calor es simple computacionalmente, no ha sido en vano, ya que, pese a que se haya podido ejecutar este código en un tiempo razonable, a continuación se verá que ya no se podrán utilizar mallas arbitrariamente grandes.

Esto conecta con los desarrollos de convergencia mostrados en el **Capítulo 3**, donde se vio que el error decrece muy rápidamente a medida que aumentamos el tamaño de la malla. Claramente, esto es completamente inútil si no se puede ejecutar el código por limitaciones computacionales.

5.2.2 Resultados para el problema de Poisson

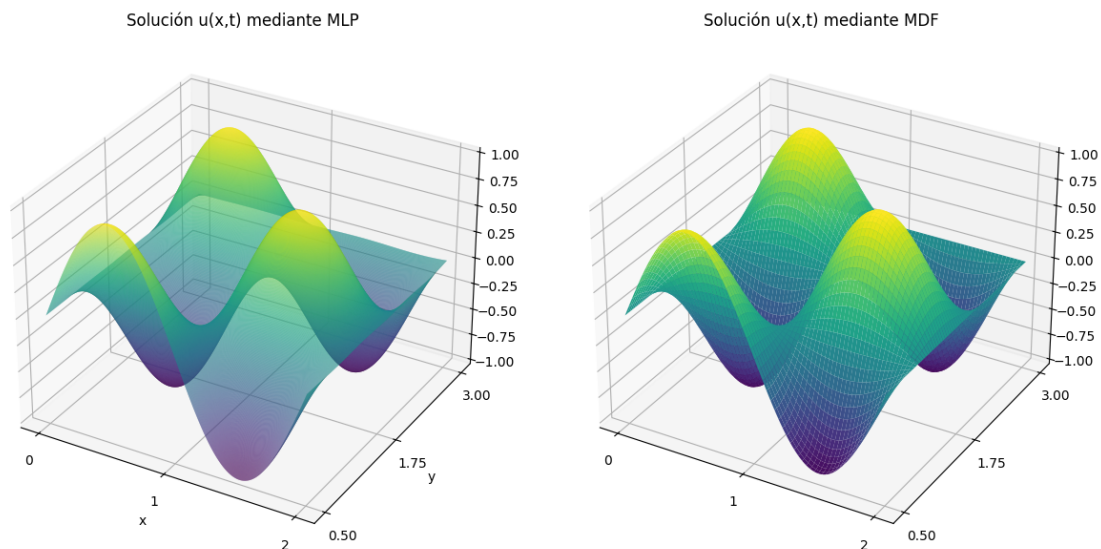


Figura 25: Soluciones aproximadas para la ecuación de Poisson.

Tanto es así que, al quitar tantos puntos de la malla, se empiezan incluso a notar diferencias a simple vista en la figura 25. Para la ecuación de Poisson, un mallado de 200×200 es prácticamente la limitación del sistema utilizado para el MDF.

Lo verdaderamente relevante aquí es destacar que no es un problema de esperar más tiempo para obtener los resultados al aumentar la malla y la precisión del MDF, lo relevante es que es imposible debido a las limitaciones del ordenador utilizado. Si bien antes para la ecuación del calor, entre el entrenamiento y la evaluación de la red, el MDF era muy superior, ahora estamos comparando un tiempo de entrenamiento de apenas 21 minutos (más 1 minuto para evaluar la malla y representar, que en este caso es de 2000×2000) de la red neuronal frente a 2 minutos y medio.

Pese a que 21 minutos pueda parecer aún demasiado, cabe destacar que esto es para alcanzar la décima época de una red con 8 capas ocultas de 50 neuronas cada una. Para comparar en condiciones más justas de tiempo, en la siguiente figura 26 podemos apreciar cómo se ve la solución de la RNA tras tan solo una época de entrenamiento.

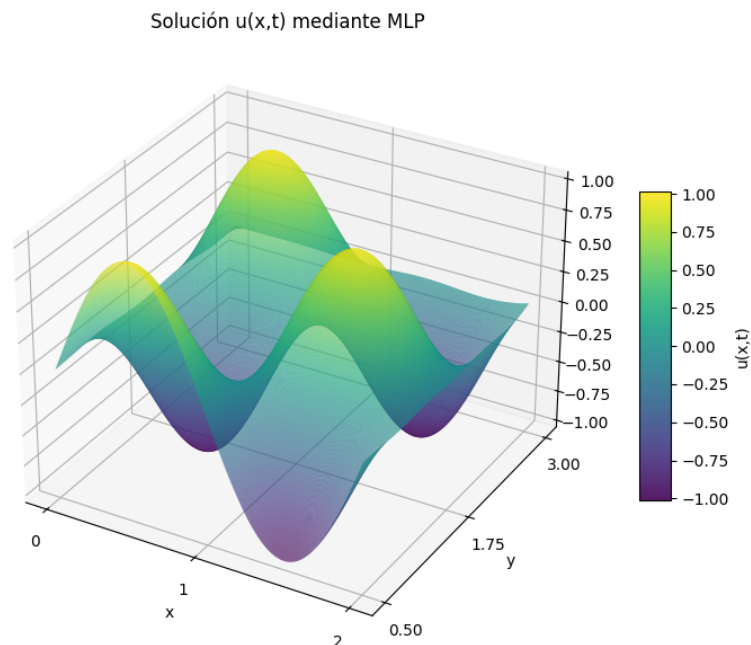


Figura 26: Solución obtenida tras 2 minutos y 6 segundos de entrenamiento, $MSE = 1.1 \times 10^{-4}$

Esto no quiere decir que no se puedan optimizar y crear mejores MDF, pero tampoco se debe olvidar que si se quiere proceder de una manera matemáticamente correcta y verificar que el método converge, se tiene que desarrollar una teoría parecida (más complicada inclusive) para aumentar el orden y, por ende, la velocidad de convergencia. Esto claramente también lleva tiempo y mucha capacidad y formación por parte de la persona que lo desarrolle.

La variación más repetida del método de 5 puntos, utilizada en este trabajo, es la del esquema de 9 puntos (basado simplemente en tomar adecuadamente más puntos de la malla para aproximar el término de orden dos) que también aparece recogido en el [5, página 64] y hay que tener en cuenta las dificultades que introducen los problemas con condiciones de contorno distintas a Dirichlet.

Por ejemplo, las condiciones de tipo Neumann necesitan tratamientos especiales para mantener el orden de convergencia, usualmente creando *nodos fantasma* (crear puntos imaginarios más allá de la frontera y apoyarse en ellos) en la ecuación de Poisson.

Y esto no es lo único interesante que tiene este problema, pues si se observa el MSE cometido por la red neuronal en todos estos puntos, este es de 1.63×10^{-6} en la malla de 2000×2000 tras las 10 épocas, frente a los 1.27×10^{-4} del MDF que también son superados por la RNA tras una época, como podemos ver en el pie de imagen 26.

Si bien, como se ha dicho, se podría mejorar un poco el método de diferencias finitas y conseguir errores similares o mejores, no deja de ser una proeza conseguir una precisión a tercer decimal y superar en un tiempo comparable a un método diseñado cuidadosamente para esta ecuación.

5.2.3 Resultados para las ecuaciones de Navier-Stokes

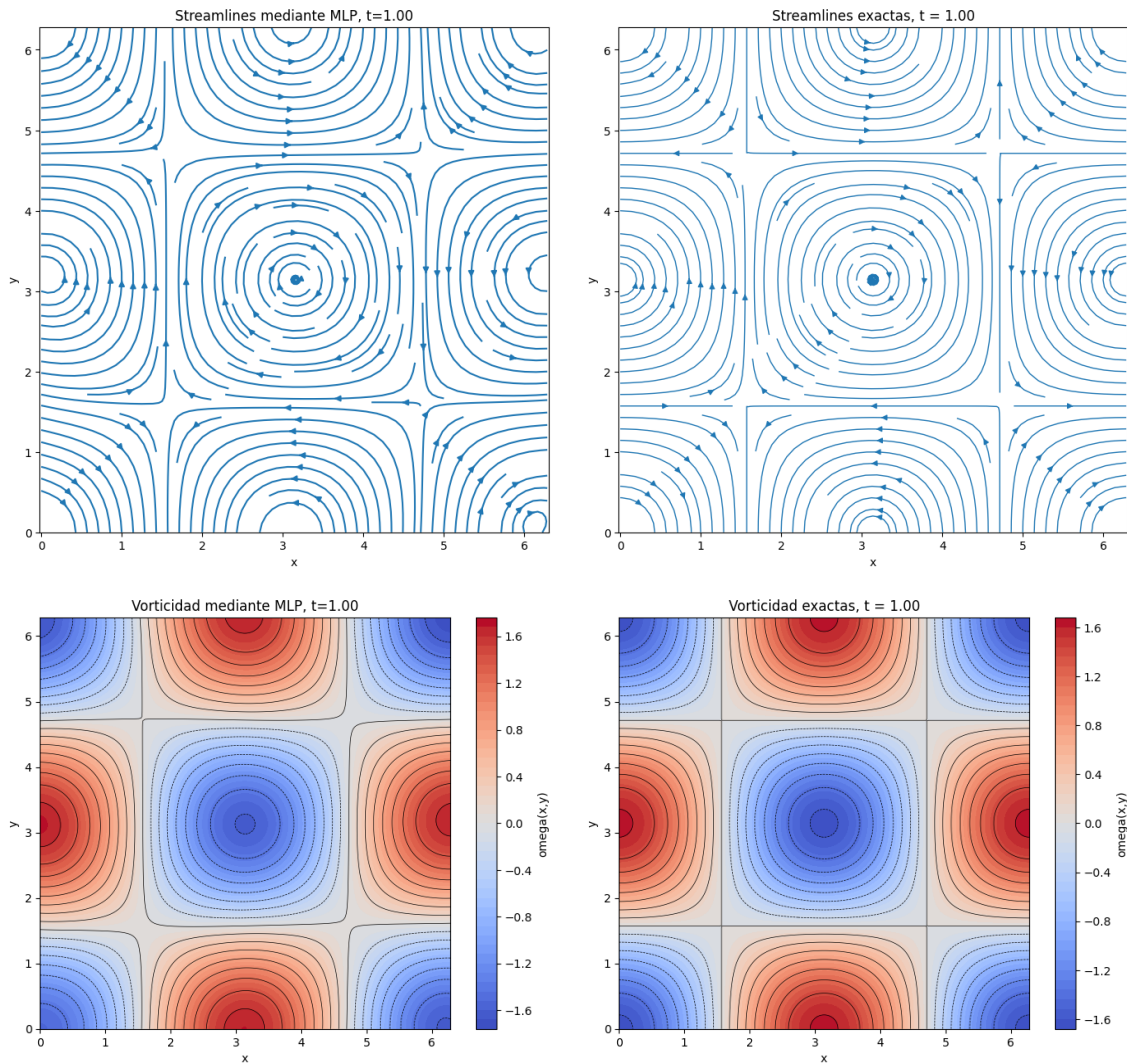


Figura 27: Soluciones aproximadas para la ecuación de Navier-Stokes.

Para esta última ecuación cabe esperar resultados incluso peores que para la ecuación de Poisson. Llegar a la solución de Taylor-Green es muchísimo más costoso computacionalmente que todo lo que se ha tratado hasta ahora. Tanto es así que en $t = 1$ se llegan a apreciar algunas diferencias claras incluso a simple vista en la figura 27 (observar las líneas de nivel que forman la cuadrícula en la derecha, como no la forman en la izquierda), incluso después de 2.5 horas de entrenamiento.

Esta red de 6 capas de 50 neuronas cada una ha sido capaz de obtener los siguientes errores en una malla espacial de 200×200 .

Tiempos (segundos)	MSE (RNA)
0.00	2.9×10^{-6}
0.25	2.2×10^{-4}
0.50	5.4×10^{-4}
0.75	7.8×10^{-4}
1.00	1.1×10^{-3}

Gracias al artículo mencionado [9] en la construcción de un MDF para este problema, se ha logrado obtener la solución de la figura 28 mediante MATLAB en la misma malla.

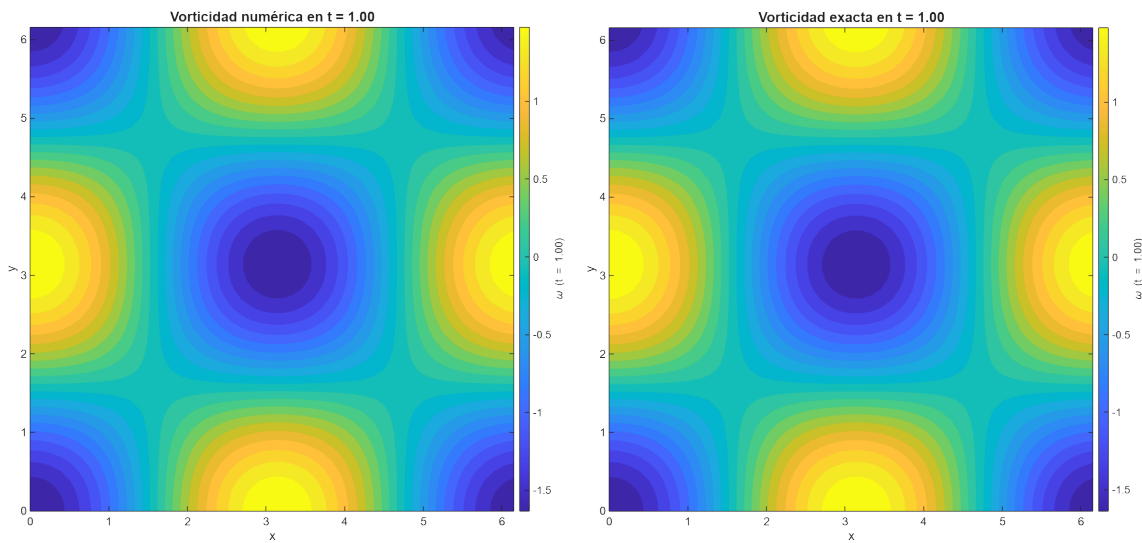


Figura 28: Resultado numérico a la izquierda, tras 440 segundos, límite técnico en 200×200 .

Como se puede apreciar, la introducción de esta viscosidad artificial mencionada anteriormente no afecta a la forma y valores de la aproximación, de forma que se pueden comparar directamente.

Utilizando un mallado de $200 \times 200 \times 200$ en espacio y tiempo, se ha obtenido un error cuadrático medio de 1.7×10^{-9} tras 7 minutos y medio, pudiéndose observar cómo aumenta el costo computacional en problemas de evolución de fluidos.

5.2.4 Ecuación del calor con α libre

En la figura 29 se presentan los resultados obtenidos de tratar el coeficiente de difusión térmica α como un parámetro más, simplemente añadiendo unas líneas de código en el mismo sentido que antes (generando valores aleatorios del coeficiente entre 0 y 2) para que la red sea capaz de dar una solución algo más general. Como se puede apreciar, los resultados obtenidos son, a simple vista, muy satisfactorios.

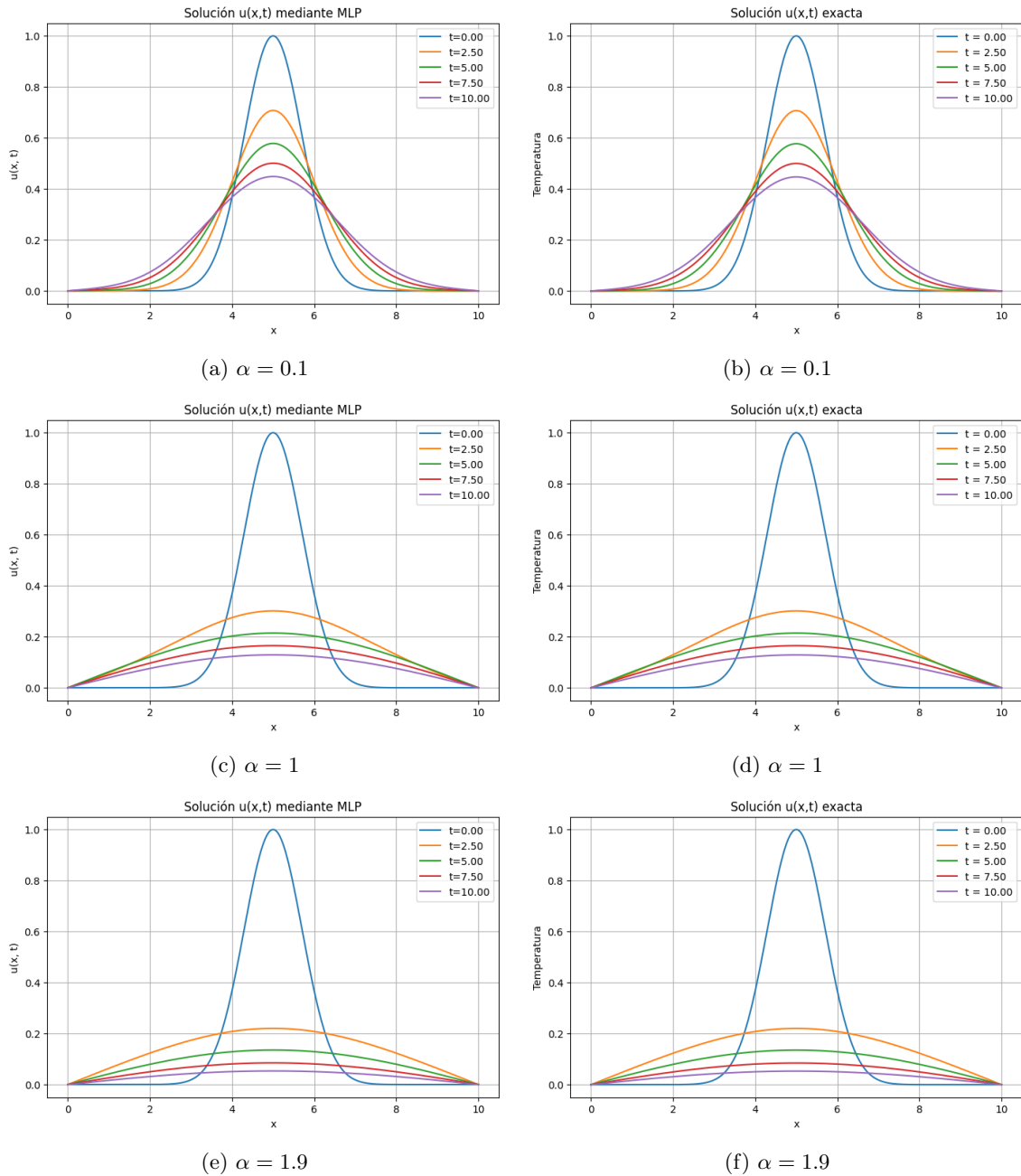


Figura 29: Comparativa entre solución aproximada y la exacta para distintos α .

Comentar sobre todo que, aparte de lo simple que es añadir este parámetro a la ecuación, para lo cual solo hay que tener cuidado de nuevo con las dimensiones de los vectores a la hora de compararlos para sacar el error, se enfatiza que el tiempo de entrenamiento no crece drásticamente con respecto al caso de α fijo ya presentado, aunque también es verdad que se partía de una red lo suficientemente amplia que no ha sido necesario ampliar. Se exponen en la siguiente tabla los errores obtenidos para diferentes tiempos tras las 50 épocas (con 10 épocas son del orden de 10^{-6}).

Tiempos	$\alpha = 0.1$	$\alpha = 1.0$	$\alpha = 1.9$
0.00	4.3×10^{-8}	2.2×10^{-8}	1.9×10^{-8}
2.50	1.0×10^{-7}	8.0×10^{-9}	1.5×10^{-8}
5.00	3.1×10^{-7}	6.7×10^{-9}	1.1×10^{-8}
7.50	4.3×10^{-7}	1.4×10^{-8}	4.1×10^{-8}
10.0	9.3×10^{-7}	1.2×10^{-8}	5.9×10^{-8}

Anteriormente se había obviado mencionar el tiempo de evaluación de la malla gigantesca de $10,000 \times 10,000$, lo cual es necesario para conocer qué método es más práctico. En ambas variantes de la ecuación del calor este tiempo es de unos 8 segundos, siendo la generación de la malla y representación de la gráfica computacionalmente inmediatas.

Inicialmente, no se podía ni tan siquiera evaluar nuestra red en una malla más pequeña, pues eran demasiados de una sola vez, pero como tanto para la parte de entrenamiento como para la de validación de redes neuronales es muy común tener este problema, actualmente es muy sencillo y cómodo dividir el conjunto en **batches** o lotes para solventar este problema.

También es interesante conocer que, si bien el tiempo de evaluación de 63 segundos mencionado en la ecuación de Poisson parece alto, si ejecutamos el código completo de `poisson_2D_NN.ipynb` veremos que entre la evaluación de la RNA y de la solución exacta allí definida (función estándar en **Python**) difiere en apenas 8 segundos, por lo que las redes presentadas, pese a tener tantas capas y neuronas, se quedan muy cerca de los tiempos de evaluación que se podrían considerar como óptimos.

5.2.5 Ecuación del calor en un dominio triangular

En este último apartado se pretenden mostrar los resultados obtenidos en una malla un tanto exótica, para lo cual se hace necesario definir el concepto de **máscaras**, que no son más que filtros modificables por el usuario que permiten seleccionar únicamente los puntos que estén en una determinada región. En este caso concreto, se definen las tres rectas que definen el triángulo de la figura 19 y se caracteriza la pertenencia a este mediante desigualdades con respecto a estas rectas, siendo el método de las máscaras muy generalizable para cualquier dominio.

Primeramente, en la figura 30 podemos ver la evolución del error para el único modelo entrenado para esta variante.

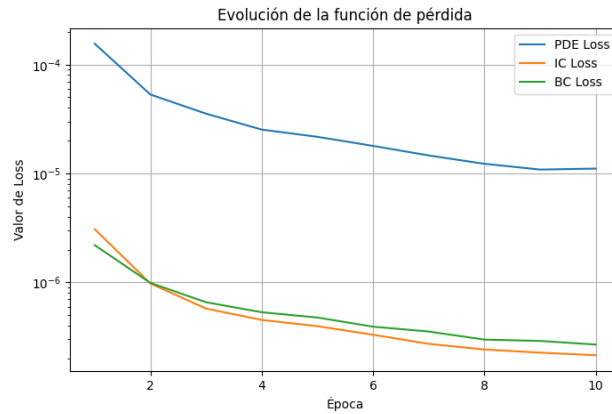


Figura 30: · Modelo 1 - 152 segundos/época

Entrenar esta red costó un total de 25 minutos, frente a los 8 segundos que tardó el solver `solvepde` comentado anteriormente de MATLAB. En la figura 31 siguiente se puede observar una comparativa visual entre ambas aproximaciones.

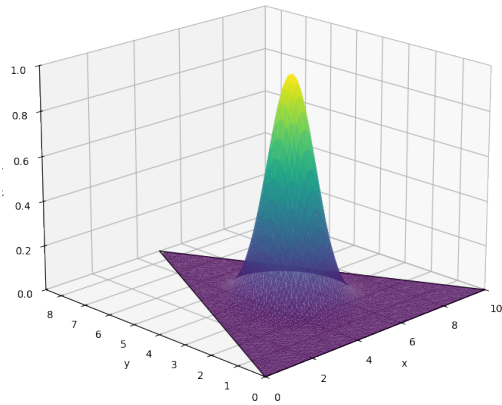
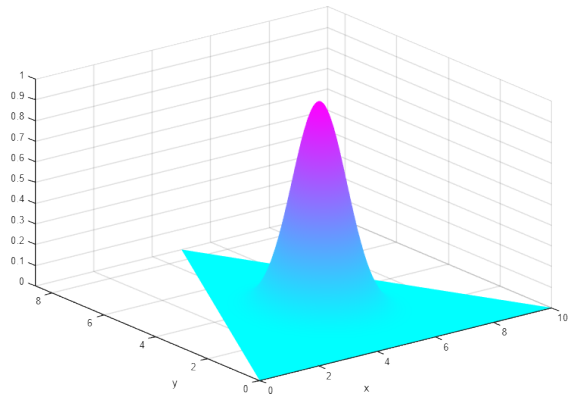
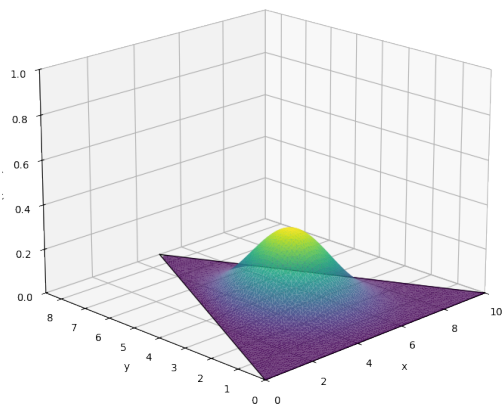
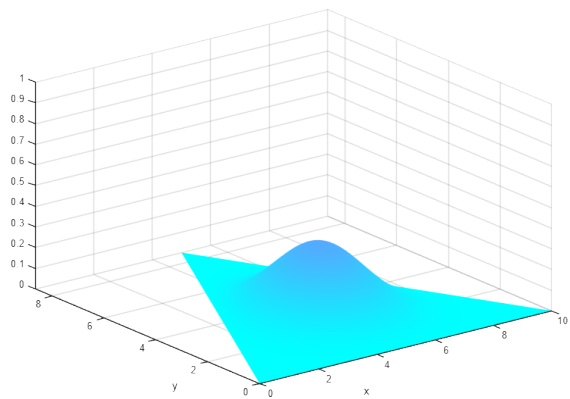
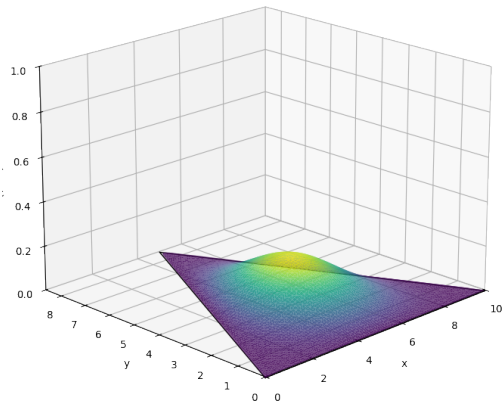
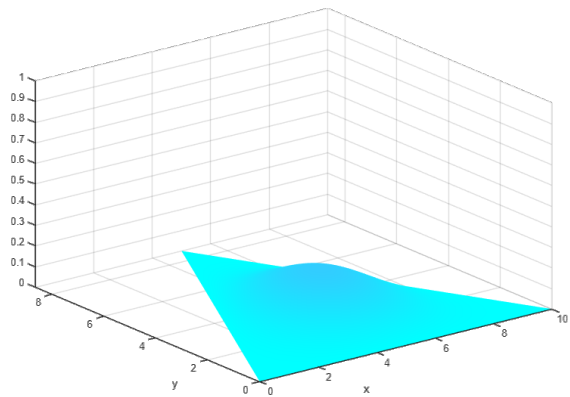
(a) Solución $u(x, y, t = 0.0)$ mediante RNA(b) Solución $u(x, y, t = 0.0)$ mediante solvepde(c) Solución $u(x, y, t = 0.5)$ mediante RNA(d) Solución $u(x, y, t = 0.5)$ mediante solvepde(e) Solución $u(x, y, t = 1.0)$ mediante RNA(f) Solución $u(x, y, t = 1.0)$ mediante solvepde

Figura 31: Comparativa entre las soluciones aproximadas por RNA y por el solver solvepde.

Las discrepancias entre ambos métodos, comparados en la malla que utilizó `solvepde`, se presentan en la siguiente tabla.

Tiempos (segundos)	MSE entre <code>solvepde</code> y la RNA
0.00	2.2×10^{-7}
0.50	4.7×10^{-8}
1.00	1.0×10^{-7}

Tabla 1: Se ha observado que, en general, cerca de los extremos del dominio definido se obtienen unos errores algo peores que en el interior. Esto no está asociado a una tendencia de incremento del error con el tiempo.

En el notebook `heat_2D_NN.ipynb` ubicado en [Github](#) se presentan además mapas de calor con la distribución de los errores absolutos entre ambos, observando que estos no van más allá de discrepancias a tercer decimal, lo cual lleva a concluir que el método seguido para entrenar la RNA es bastante robusto.

5.3 Discusión

Esta sección se utilizará para analizar y resumir los contenidos presentados a lo largo del documento, sin quitarle espacio o importancia a la conclusión que se presenta en el último Capítulo. Se tratarán de definir una serie de virtudes e inconvenientes claros para cada método y mostrar los campos en los que las RNAs pueden tener cabida en términos de resolver ecuaciones de derivadas parciales.

El primer punto a comentar es la diferencia en complejidad a la hora de definir la estructura o algoritmo. Para las redes neuronales, se necesitan unas nociones relativamente básicas de Python y ser capaz de transcribir el sistema de ecuaciones dentro del bloque de entrenamiento, utilizando la estructura ya vista y que no ha cambiado a lo largo del trabajo.

En este sentido, el esfuerzo para desarrollar y codificar un método de diferencias finitas es mayor y requiere de más conocimientos, tanto de matemáticas como de recursos informáticos, para asegurar la convergencia y conseguir poder almacenar todos los objetos necesarios en nuestro sistema durante la ejecución del código.

El caso en el que se comienza a apreciar esta diferencia es en el pequeño problema de dinámica de fluidos tratado, pues incluso se ha debido recurrir a un artículo de apoyo con

ciertas indicaciones para construir el MDF mientras que para la red neuronal solo se han tenido que adaptar el código de manera natural.

En esta línea, cabe hacer una reflexión sobre la complejidad verdadera de la convergencia de los métodos. Si bien es verdad que ya se han presentado casos concretos del desarrollo teórico que hay que realizar para el MDF, no se ha comentado gran cosa de los desarrollos teóricos necesarios para asegurar la convergencia de los perceptrones multicapa per se. Si bien se ha presentado el teorema de aproximación universal, los conocimientos necesarios para conseguir demostrarlo rigurosamente requieren un esfuerzo matemático que, desde luego, no es simple.

La diferencia aquí radica en que conocer la teoría completa de los métodos en diferencias finitas es indispensable si se trabaja con una amplia variedad de EDPs, ya que esta teoría son las técnicas que luego se utilizan en ecuaciones nunca vistas y se necesitan para garantizar la convergencia en tiempos asumibles.

De igual forma, desde el punto de vista de las redes neuronales, también se deben conocer técnicas para construirlas y saber muy bien cómo funciona cada elemento de las mismas, pues no se dispone de un resultado o procedimiento exacto que permita conocer el tamaño de la red o qué hiperparámetros se deben utilizar. Esto sin contar que ciertas ecuaciones con propiedades particulares, como discontinuidades, no pueden ser resueltas con la red neuronal presentada, haciendo necesario desarrollar arquitecturas nuevas.

Otro detalle a destacar es la velocidad de convergencia, lo bien que progresan los métodos. Si bien las redes neuronales consiguen muy buenos resultados, hasta cierto punto podrían ser incluso sorprendentes, siempre que se consiga definir un MDF de orden 2 o superior y se pueda utilizar una malla lo suficientemente grande, se obtendrá un mejor tiempo y precisión debido a que los métodos en diferencias finitas se diseñan específicamente para resolver una ecuación particular. De nuevo, esto quiere decir que, si se dispone del tiempo, los recursos y los conocimientos adecuados, actualmente es preferible utilizar los MDFs.

Seguidamente, se mencionará un factor muy importante sobre las RNAs, el cual es su capacidad para poder trabajar progresivamente. Si se presta atención al código presentado, al inicio del bucle de entrenamiento hay un bloque que permite continuar el entrenamiento

de la red desde donde se dejó, habiéndose guardado tanto los parámetros de la red como el número de pasos utilizados para entrenarla desde ese mismo punto.

Además, se dispone de un sistema que va mostrando las métricas de error en tiempo real, de forma que rápidamente se pueda ver si hay algo mal configurado o si se debería modificar algún hiperparámetro. Si bien no es una solución al problema antes mencionado sobre el desconocimiento de la configuración exacta de la red para asegurar convergencia, sí que hace el proceso de entrenamiento más amigable y moldeable sobre la marcha, dejando claro que no hay que completar todo el proceso de entrenamiento para descubrir si la red funciona.

Por último, no se puede obviar la libertad de evaluación a lo largo de todo el dominio de entrenamiento, cualidad que ya ha sido ampliamente comentada, ni tampoco de la comodidad a la hora de almacenar los resultados. Los métodos tradicionales están obligados a guardar en memoria todos los resultados obtenidos, los cuales pueden llegar a ocupar bastante memoria dependiendo de la malla. Esto puede resultar en un problema de almacenamiento en algunos casos, por ejemplo si se trabaja mucho con este tipo de ecuaciones. Las redes neuronales ofrecen la alternativa de guardar la propia red en un fichero del orden de megabytes, que solo dependen de la arquitectura de la red y no de su precisión.

En resumen, las redes neuronales ofrecen soluciones fácilmente almacenables y amigables que permiten al usuario realizar simulaciones de fenómenos concretos con una alta precisión, bajos tiempos de evaluación y pocos recursos. Tanto es así que podría llegar a ser interesante verlas aplicadas en entornos de realidad virtual, videojuegos o páginas web, para conseguir, por ejemplo, mayor naturalidad en interacciones con fluidos, las cuales todavía dan muchos problemas en estos tres campos.

Lo mejor de todo es que aún hay muchísimo margen de mejora y avance en el terreno del machine learning, en el cual se están realizando grandes inversiones y está ganando influencia en el desarrollo de las nuevas tecnologías, habiéndose duplicado el rendimiento en las librerías utilizadas durante el trabajo en apenas dos generaciones de tarjetas gráficas comerciales [12].

Sin embargo, en proyectos a gran escala en los que se disponga de muchos recursos, como superordenadores, y se requiera de muy alta fiabilidad, los métodos tradicionales son los más utilizados y estudiados y, muy posiblemente, lo seguirán siendo. Esto incluye a la meteorología, la astrofísica y en la aquí presentada dinámica de fluidos. Por tanto, no se debe cuestionar la gran importancia que estos tienen ni rechazarlos, ya que son el estándar de muchas áreas importantes.

Capítulo 6

Conclusiones

A lo largo de este Trabajo Fin de Grado se ha profundizado en los fundamentos teóricos de las ecuaciones en derivadas parciales (EDP) y sus aplicaciones numéricas y mediante aprendizaje automático. Para los primeros, los cuales fueron introducidos a lo largo de la titulación, se realizó un análisis profundo y riguroso de la teoría que los fundamenta y de su implementación, siendo útiles para interiorizar estos conocimientos. Para los segundos no se disponía de un conocimiento previo, de forma que la introducción a los conceptos de inteligencia artificial aquí presentados han servido como inicialización en la materia, saciando la curiosidad de la que se partía en un inicio.

Ponerlos a prueba en el terreno de la resolución de las ecuaciones en derivadas parciales ha permitido sacar comparativas con respecto al MDF, el cual fue específicamente creado para este terreno, obteniendo resultado muy satisfactorios y demostrando que actualmente pueden incluso tener utilidad en algunas áreas, mayormente vinculadas a la informática. También han permitido observar fenómenos como el aumento del tiempo de entrenamiento al aumentar la dimensionalidad del problema y observar como la complejidad de la ecuación afecta a la precisión de la red, haciendo evidente la necesidad de tener que proceder en la mayoría de casos mediante prueba y error a la hora de definir la red neuronal.

Igualmente, la flexibilidad mostrada en términos de incorporación de parámetros del problema, como coeficientes físicos, y de geometría del dominio, han permitido comprobar que existen ciertos escenarios en los que la arquitectura presentada puede superar en utilidad a los métodos clásicos.

En definitiva, esta comparación pone de manifiesto que no existe un método universalmente superior, sino que la elección depende, como casi todo en la vida, del contexto.

Bibliografía

- [1] Lokenath Debnath Tyn Myint-U. *Linear Partial Differential Equations for Scientists and Engineers*. 4.^a ed. Boston: Birkhäuser Boston, dic. de 2006.
- [2] Richard Haberman. *Elementary Applied Partial Differential Equations: With Fourier Series and Boundary Value Problems*. 2.^a ed. Englewood Cliffs, NJ: Prentice-Hall, 1987. ISBN: 9780132528757.
- [3] Aslak Tveito y Ragnar Winther. *Introduction to Partial Differential Equations. A Computational Approach*. Vol. 29. Texts in Applied Mathematics. Paperback reprint of the 2005 edition. Berlin: Springer-Verlag, 2009. ISBN: 978-3642002953.
- [4] Andrea L. Bertozzi Andrew J. Majda. *Vorticity and Incompressible Flow*. 1.^a ed. Cambridge: Cambridge University Press, nov. de 2001.
- [5] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations. Steady-State and Time-Dependent Problems*. Philadelphia, PA: Society for Industrial y Applied Mathematics (SIAM), 2007. ISBN: 978-0898717839.
- [6] Zhilin Li, Zhonghua Qiao y Tao Tang. *Numerical Solution of Differential Equations. Introduction to Finite Difference and Finite Element Methods*. Cambridge: Cambridge University Press, 2018. ISBN: 978-1108425203.
- [7] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. ISBN: 9780262035613. URL: <https://www.deeplearningbook.org/>.
- [8] Sarah Guido Andreas C. Müller. *Introduction to Machine Learning with Python*. Ed. por Dawn Schanafelt. 2.^a ed. California: O'Reilly, jun. de 2019.
- [9] Endalew Getnet Tsega y Oluwole D. Makinde. «Advancing Finite Difference Solutions for Two-Dimensional Incompressible Navier–Stokes Equations Using Artificial Compressibility Method and Sparse Matrix Computation». En: *Journal of Applied Mathematics* 2024 (ago. de 2024), págs. 1-9. DOI: 10.1155/2024/5506715. (Visitado 27-05-2025).
- [10] Vincent Sitzmann et al. «Implicit Neural Representations with Periodic Activation Functions». En: *arXiv preprint arXiv:2006.09661* (jun. de 2020). DOI: 10.48550/arXiv.2006.09661. arXiv: 2006.09661 [cs.CV]. (Visitado 27-05-2025).
- [11] The MathWorks, Inc. *Partial Differential Equation Toolbox*. R2024b. The MathWorks, Inc. Natick, Massachusetts, 2024. URL: <https://es.mathworks.com/help/releases/R2024b/pde/index.html> (visitado 27-05-2025).
- [12] AIME Team. *Deep Learning GPU Benchmarks 2024: NVIDIA H100, A100, L40S, RTX 6000, 3090, 4090, A6000*. AIME Blog. 2024. URL: <https://www.aime.info/blog/en/deep-learning-gpu-benchmarks/> (visitado 25-05-2025).

- [13] Universidad de La Laguna. *Teorema de Stokes y Gauss. Tema 3, Sección 12.* n.d. URL: https://campusvirtual.ull.es/ocw/pluginfile.php/3964/mod_resource/content/3/tema3/12-tstokesgauss.pdf (visitado 27-05-2025).
- [14] Cosmic Wanderer. *Introduction To Navier-Stokes Equations.* 4 min read. Math Simplified en Medium. 19 de nov. de 2021. URL: <https://medium.com/math-simplified/introduction-to-navier-stokes-equations-f32d9d763a60> (visitado 27-05-2025).
- [15] Nenad Miljkovic. *Lecture 2 – Fouriers Law of Heat Conduction.* Lecture notes for ME 320: Heat Transfer, Energy Transport Research Lab, University of Illinois at Urbana-Champaign. 2017. URL: <https://etrl.mechanical.illinois.edu/pdf/ME320/Lecture%20%20-%20Fouriers%20Law%20of%20Heat%20Conduction.pdf> (visitado 27-05-2025).
- [16] *Introducción a las redes neuronales informadas por la física (PINN).* MathWorks. URL: <https://es.mathworks.com/discovery/physics-informed-neural-networks.html> (visitado 27-05-2025).
- [17] Yarin Gal, Jiří Hron y Alex Kendall. «Concrete Dropout». En: (2017). arXiv: 1708.07469 [cs.LG]. URL: <https://arxiv.org/abs/1708.07469> (visitado 27-05-2025).

Glosario

A continuación, se presentan los términos relevantes utilizados a lo largo de este TFG junto a la página donde aparecen por primera vez.

Teoría Fundamental de EDPs

Ecuación en Derivadas Parciales (EDP), [4](#)

Operadores ∇ y Δ , [5](#)

Solución, [5](#)

Orden de una ecuación, [5](#)

Ecuación lineal, [5](#)

Ecuación homogénea, [5](#)

Condiciones iniciales (IC), [6](#)

Condiciones de contorno (BC), [7](#)

Ecuación del calor (heat equation), [9](#)

Flujo, [9](#)

Ecuación de Poisson, [14](#)

Fluido, [17](#)

Fluido incompresible, [17](#)

Viscosidad, [20](#)

Ecuaciones de Navier-Stokes, [20](#)

Vorticidad, [20](#)

Vórtice de Taylor–Green, [22](#)

Streamfunction y streamlines, [22](#)

Formalización del MDF

Método de Diferencias Finitas (MDF), [25](#)

Serie de Taylor, [25](#)

Malla, [25](#)

Nodos, [25](#)

Error de consistencia, [26](#)

Norma del supremo, [26](#)

Error discreto, [26](#)

Estabilidad, [26](#)

Método explícito, [29](#)

Método implícito, [29](#)

Norma euclídea, [32](#)

Error Cuadrático Medio (MSE), [34](#)

Producto de Kronecker, [37](#)

Operación vec, [37](#)

Conceptos Básicos de ML

Aprendizaje Automático (ML), [43](#)

Modelo, [43](#)

Función de pérdida, [44](#)

Pasos y épocas, [44](#)

Red Neuronal Artificial (RNA ó NN), [44](#)

Parámetros y hiperparámetros, [44](#)

Perceptrón, [44](#)

Tasa de aprendizaje, [45](#)

Perceptrón Multicapa (MLP), [45](#)

Función de activación, [46](#)

ReLU, [46](#)

Retropropagación, [47](#)

Optimizador, [47](#)

Planificador, [48](#)

Extensión “solvepde”, [52](#)